

SECR2033
Computer Organization and Architecture

Module 4
Instruction Set Architecture (ISA)

Objectives:

- To provide a more detailed look at machine instruction sets.
- To look at different instruction types and operand types, and how instructions access data in memory.
- To understanding how instruction sets are designed and how their function can help to understand the more intricate details of the architecture of the machine itself.

Module 4
Instruction Set Architecture (ISA)

4.1 Introduction
4.2 Machine Instruction Characteristics
4.3 Types of Operands
4.4 Addressing Modes
4.5 Instruction Formats
4.6 Summary

Module 4
Instruction Set Architecture (ISA)

4.1 Introduction

- Overview
- Design Decisions for Instruction Sets
- ISA Level
- General Concepts:
  x86 Processor Architecture

4.2 Machine Instructio
4.3 Types of Operand
4.4 Addressing Mode
4.5 Instruction Format
4.6 Summary

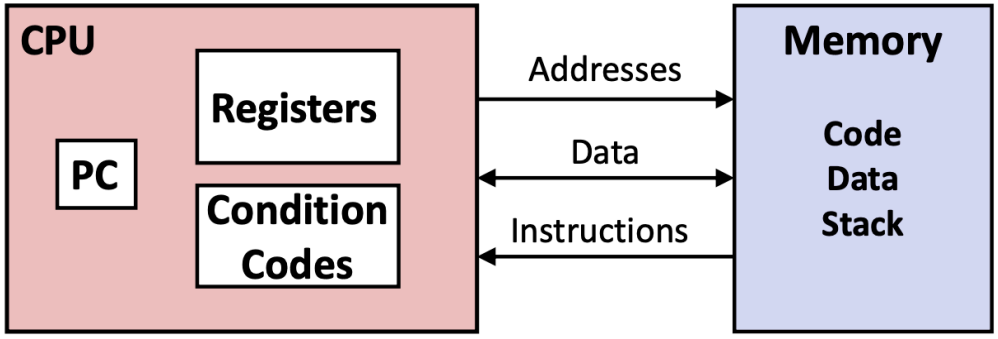
4 Overview

ALU (Arithmetic Logic Unit)

- One boundary where the computer designer and the computer programmer can view the same machine is the machine instruction set.
- Implementing the processor is a task that in large part involves implementing the machine instruction set.
- The user who chooses to program in machine language (actually, in assembly language) becomes aware of the register and memory structure, the types of data directly supported by the machine, and the functioning of the ALU.

Assembly Programmer's View

- Programmer-Visible State
  - PC
    - = program counter
    - Address of next instruction
    - Called "EIP" (x86)
  - Registers
    - Heavily used to store program data
  - Condition codes
    - Store status information about most recent arithmetic operation
    - Used for conditional branching



- Memory
  - Byte addressable array
  - Code and user data
  - Stack to support procedures

4 Design Decisions for Instruction Sets

- When a computer architecture is in the design phase, the instruction set format must be determined before many other decisions can be made.
- Selecting this format is often quite difficult because the instruction set must match the architecture.
- If the architecture is well designed, it could last for decades.

What is an Instruction Set?

- The complete collection of instructions that are understood by a CPU
- Known also as Machine Code / Machine Instruction
- Binary representation
- Usually represented by assembly codes
- Part of a processor design that one needs to understand to write assembly code.

- Instruction Set Architectures (ISAs) are measured by several different factors, including:

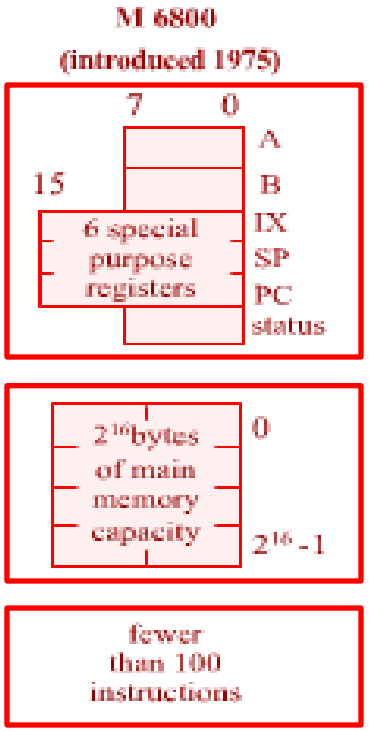
(1) the amount of space a program requires;

(2) the complexity of the instruction set;

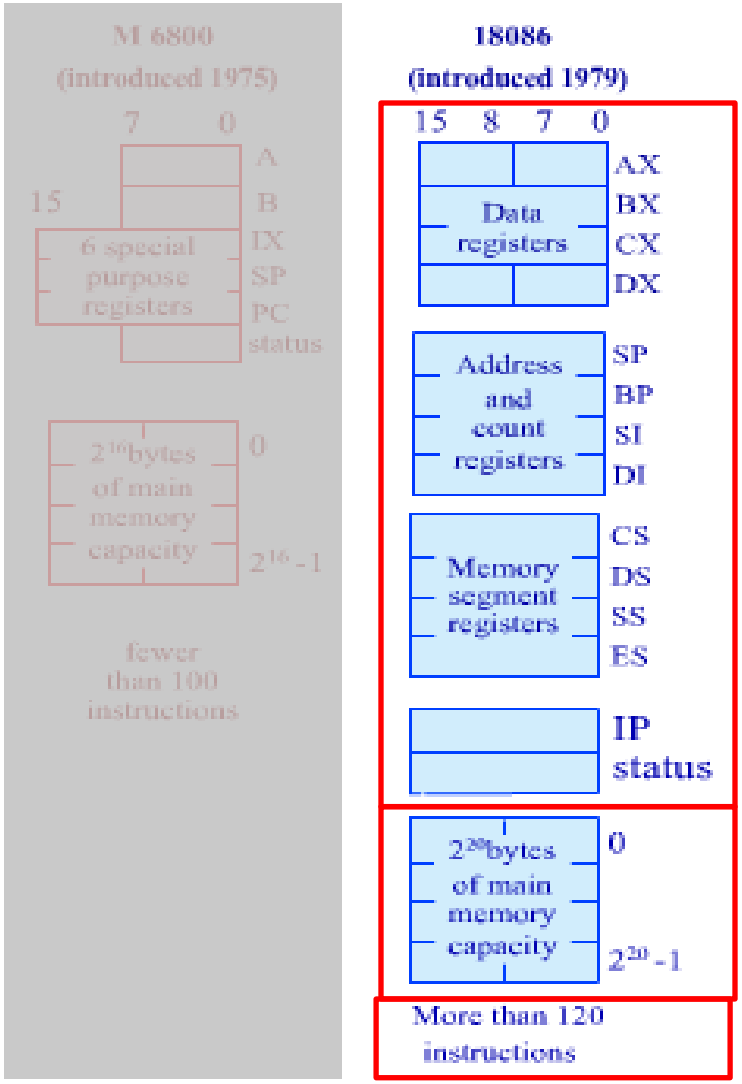
(3) the length of the instructions; and

(4) the total number of instructions.

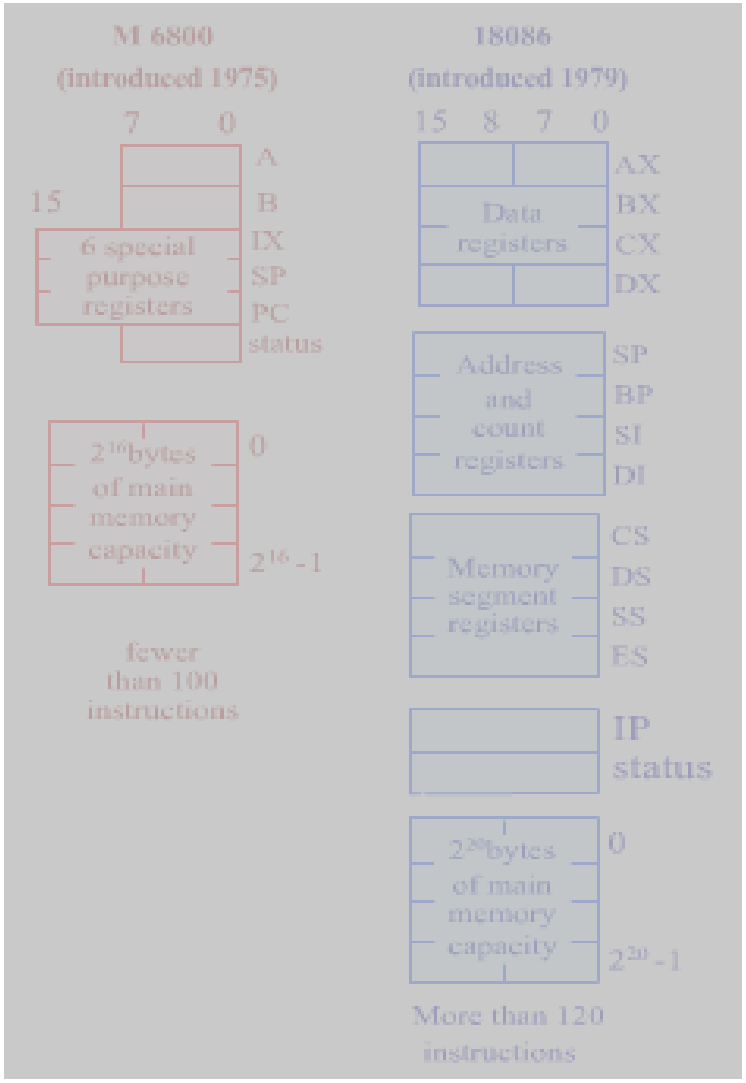
Example of some ISA



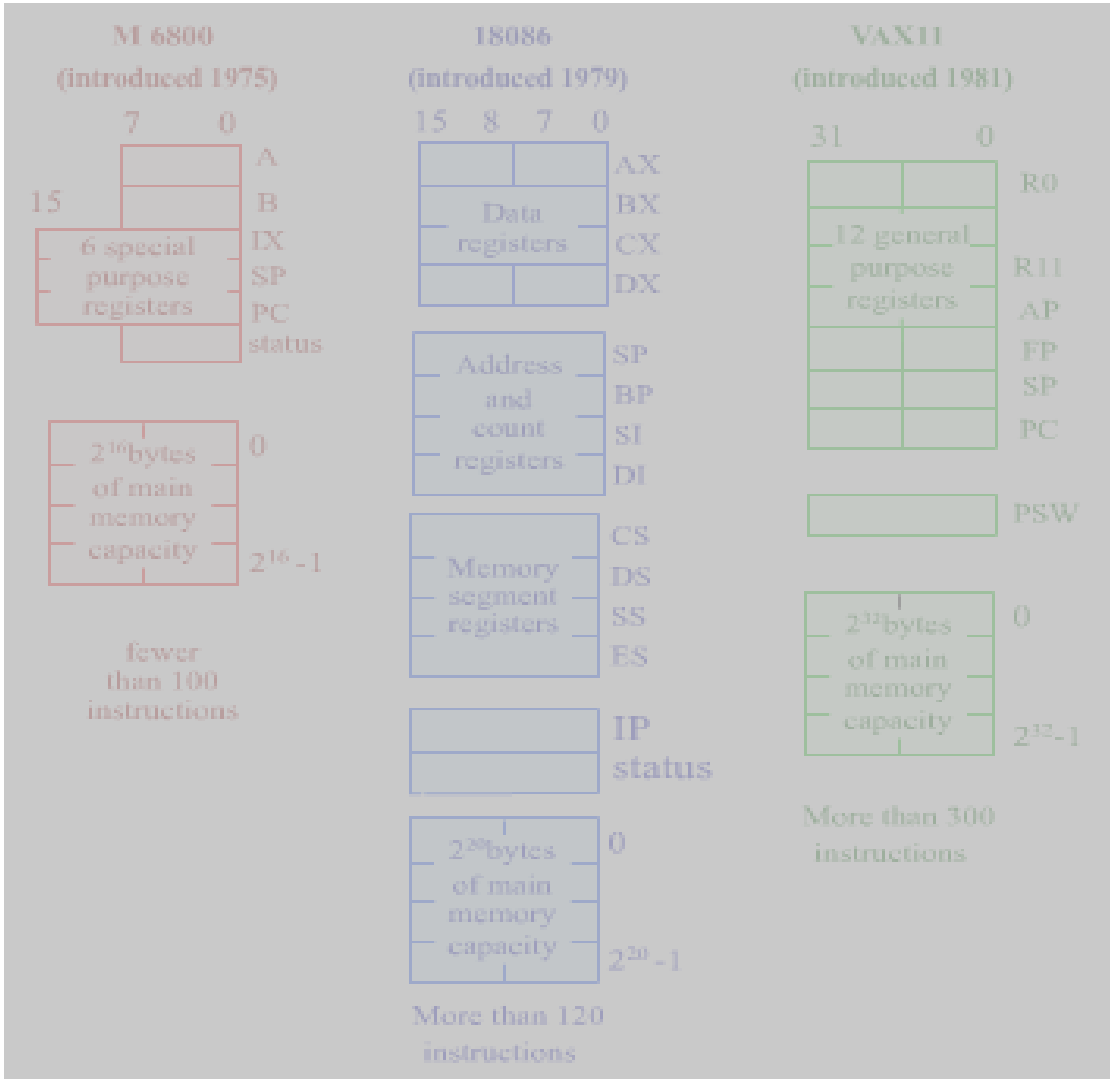
Example of some ISA



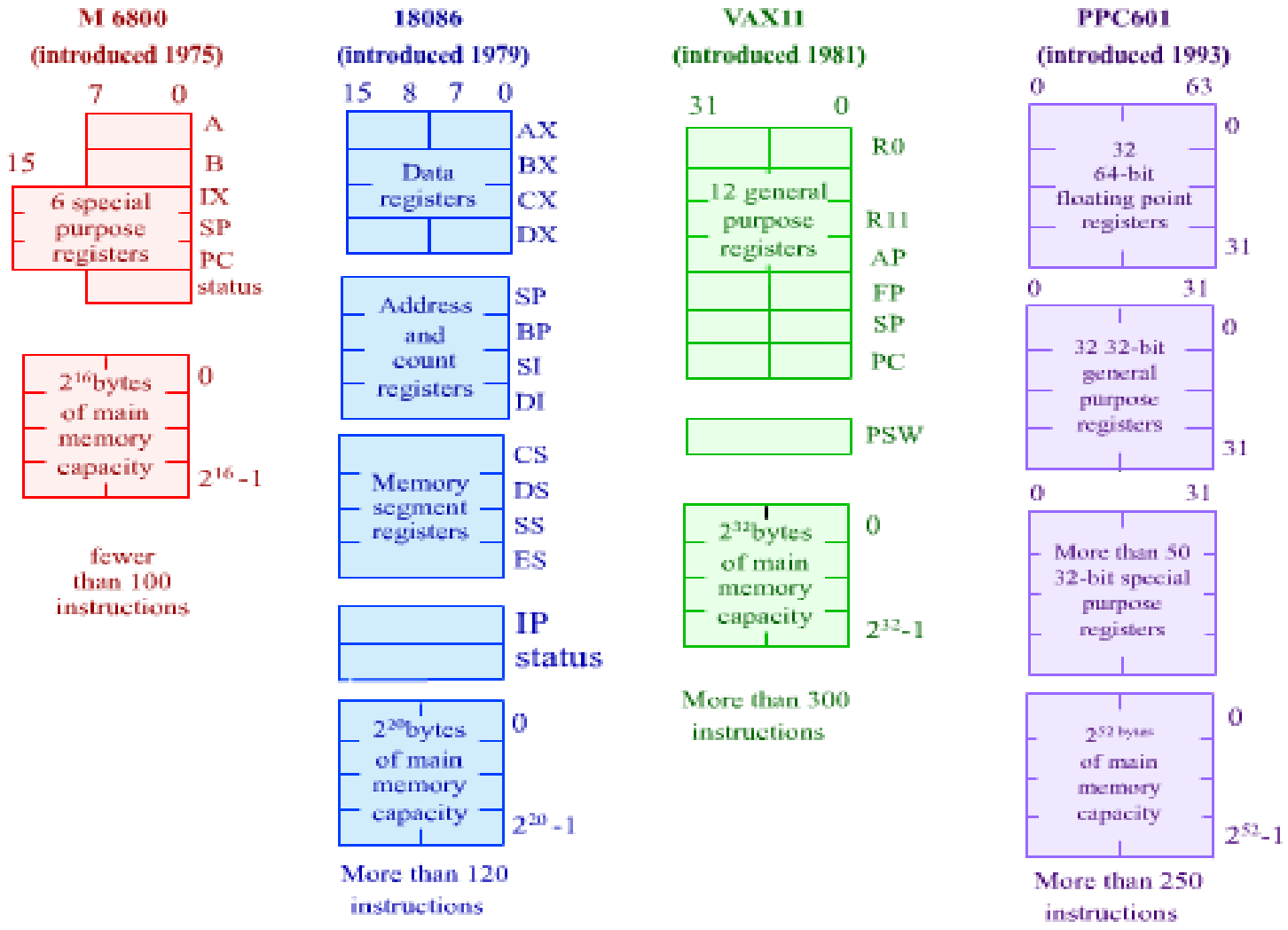
Example of some ISA



Example of some ISA



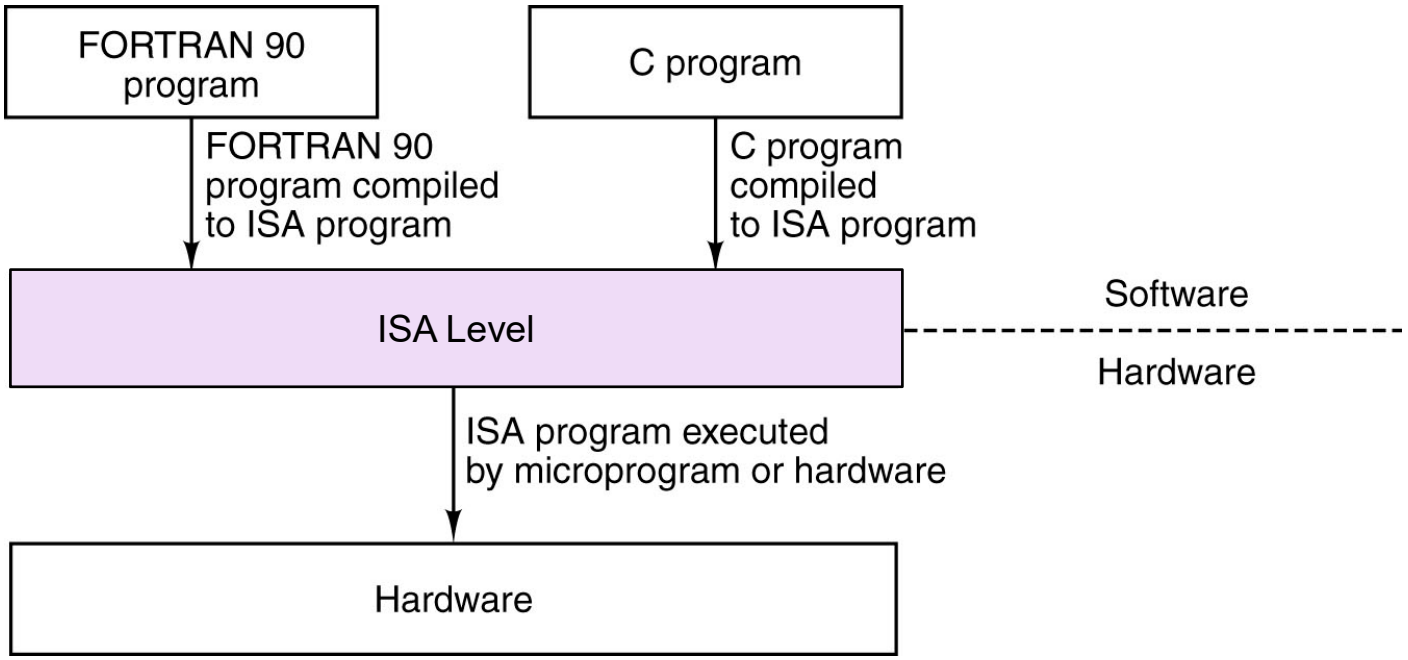
Example of some ISA



ISA Level

ISA (Instruction Set Architecture)

- ISA Level defines the interface between the compilers (high level language) and the hardware. It is the language that both understand.



High-Level Languages: Advantages



Program development is faster: fewer instructions to code.



Program maintenance is easier: same reasons as above.

However, *Assembly language* programs are *not portable*.



Programs are portable:

- ❑ Contain few machine-dependent details → Can be used with little or no modifications on different machines.
- ❑ Compiler translates to the target machine language.

Why Assembly Languages?





Accessibility to system hardware:

- Assembly language is useful for implementing system software.
- Also useful for small embedded system applications.



Space and Time efficiency:

- Understanding sources of program inefficiency.
- Tuning program performance.
- Writing compact code.



Writing assembly programs gives the computer designer the needed deep understanding of the instruction set and how to design one.



To be able to write compilers for HLLs, we need to be expert with the machine language. Assembly programming provides this experience.

4

General Concepts:
X86 Processor Architecture

- This section describes the architecture of the x86 processor family and its host computer system from a programmer's point of view.
- *Assembly language* is a great tool for learning how a computer works, and it requires you to have a working knowledge of computer hardware.
- The concepts in this section will help to understand the assembly language code that be written.

Basic Microcomputer Design

(Section 5.2)

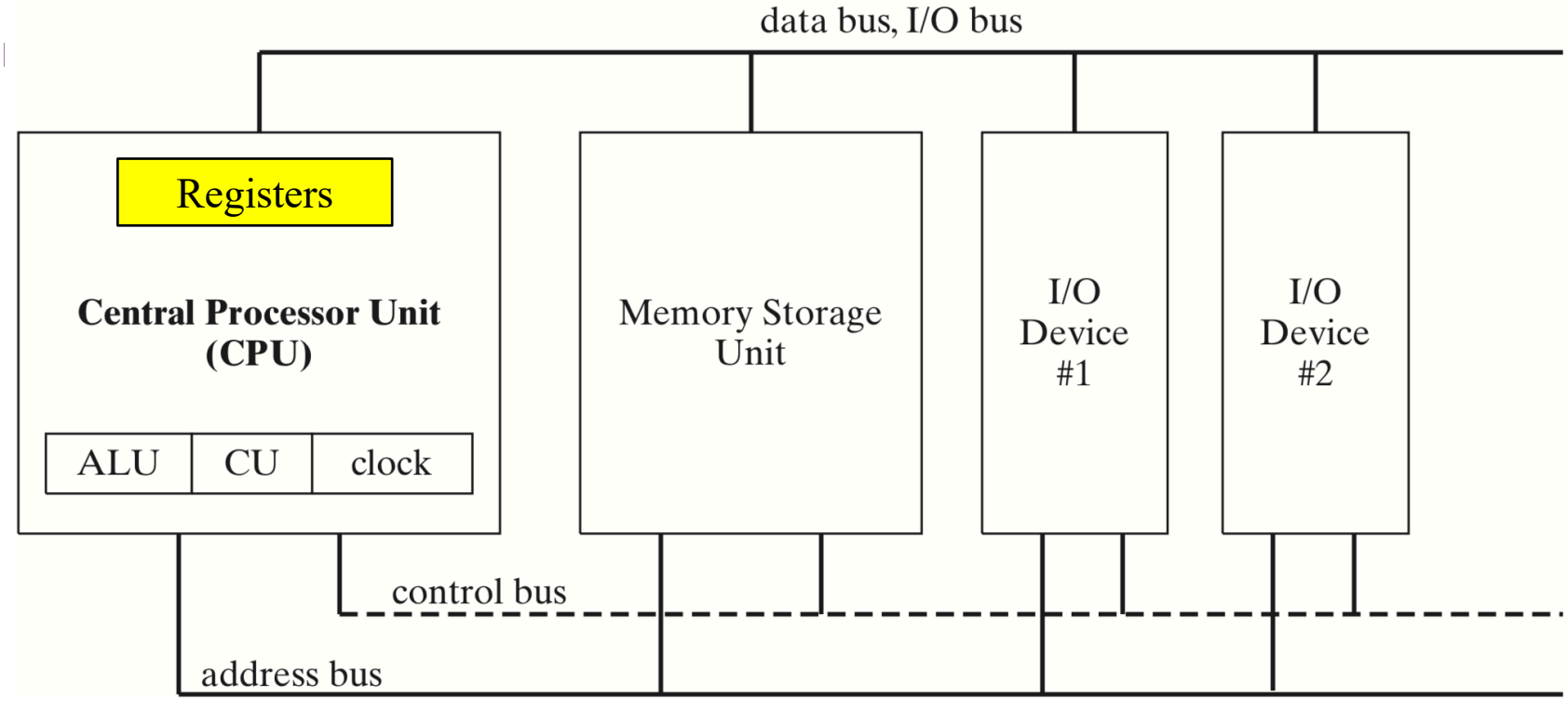


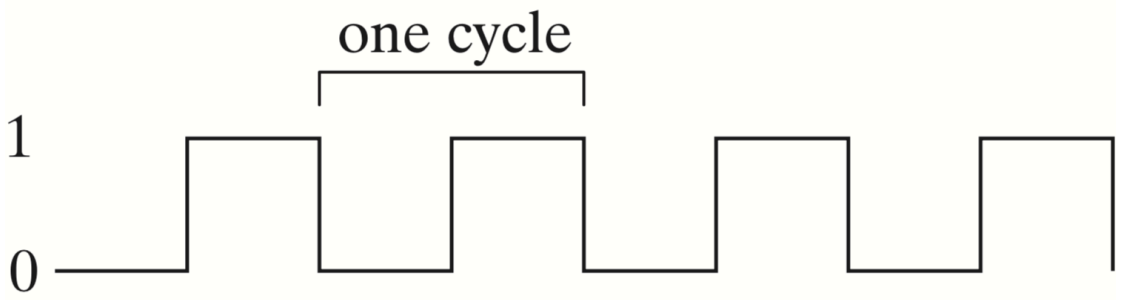
Figure: Block diagram of a microcomputer.

- The CPU does the *calculations* and *logic operations* → contains a limited number of storage locations (i.e., register), a high-frequency clock, a control unit, and an arithmetic logic unit

Elements	Description
Clock	○ synchronizes the internal CPU operations.
Control Unit (*CU*)	○ coordinates sequence of execution steps.
Arithmetic Logic Unit (*ALU*).	○ performs arithmetic and bitwise processing.

Clock

- synchronizes all CPU and BUS operations.
- machine (clock) cycle measures time of a single operation.
- clock is used to trigger events.



Instruction Execution Cycle

- Pre-defined steps to execute 1 machine instruction:
  - *Fetch*
  - *Decode*
  - *Fetch operands*
  - *Execute*
  - *Store output*

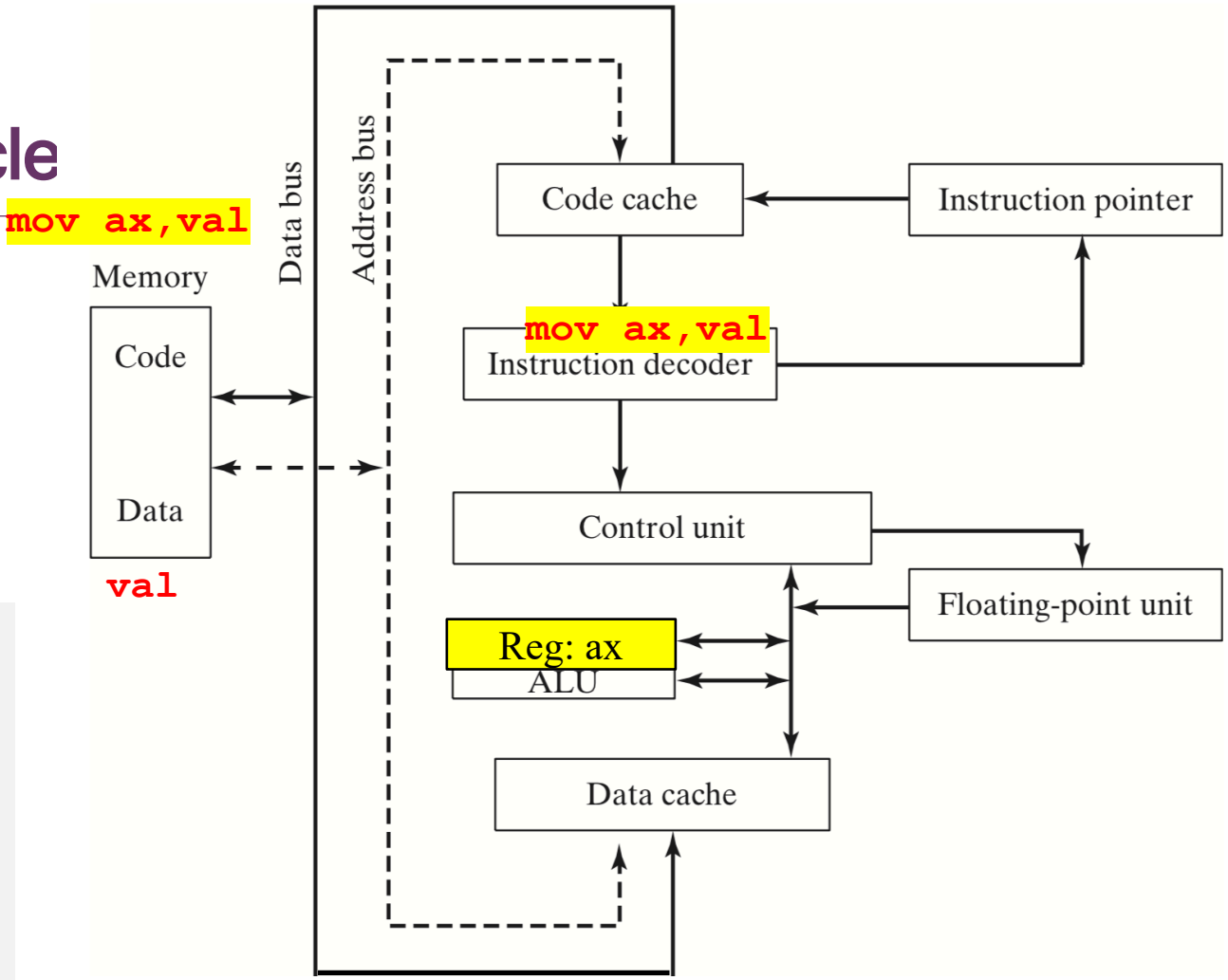


Figure: Simplified CPU block diagram.

Irvine, K.R. (2015). *Assembly Language for x86 Processors* (7th Edition). New Jersey: Pearson Education Limited, p.35.

Reading From Memory

- Multiple machine cycles are required when reading from memory, because it responds much slower than the CPU.

- The steps are:
  - **Cycle 1**: address placed on address bus (ADDR).
  - **Cycle 2**: *Read Line* (RD) set low (0).
  - **Cycle 3**: CPU waits one cycle for memory to respond.
  - **Cycle 4**: *Read Line* (RD) goes to high (1), indicating that the data is on the data bus (DATA).

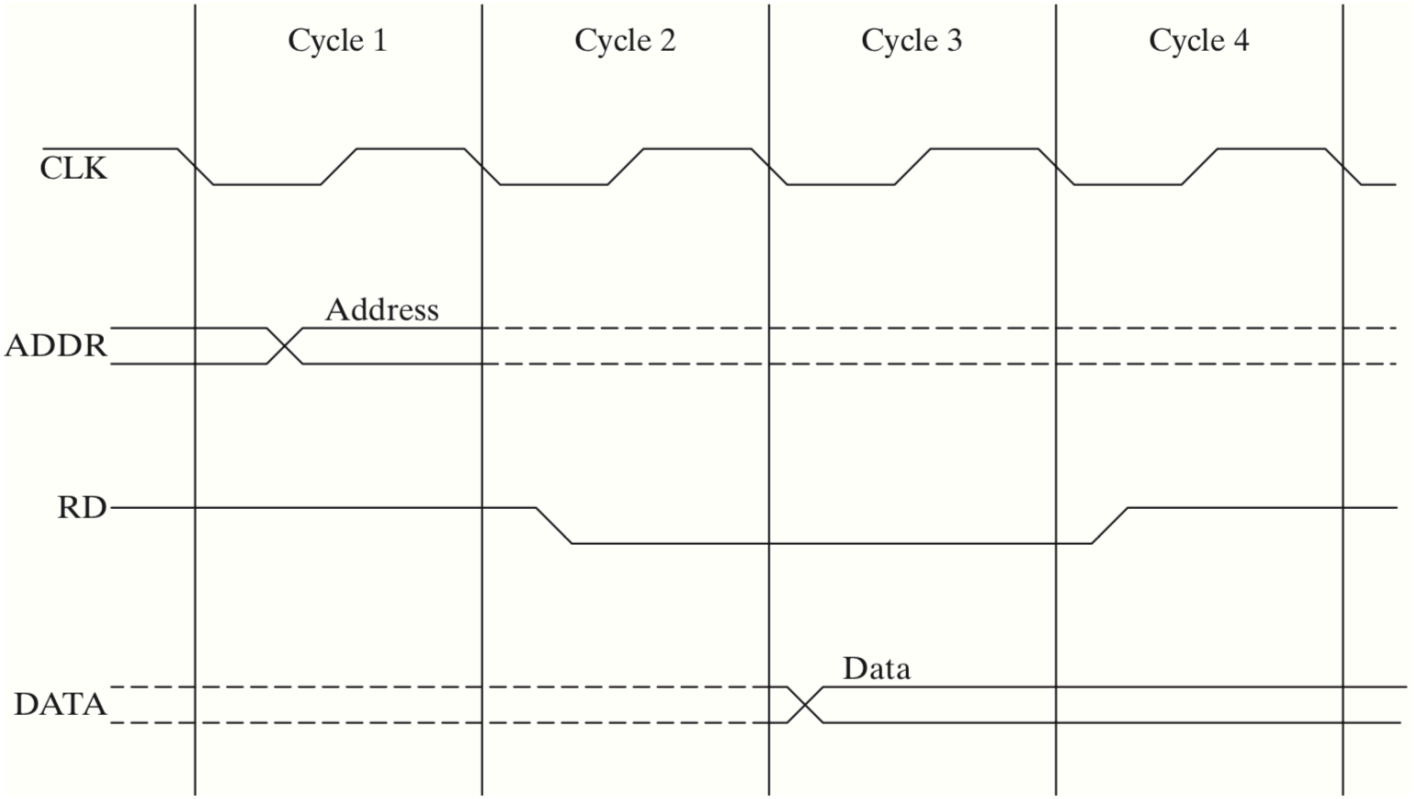
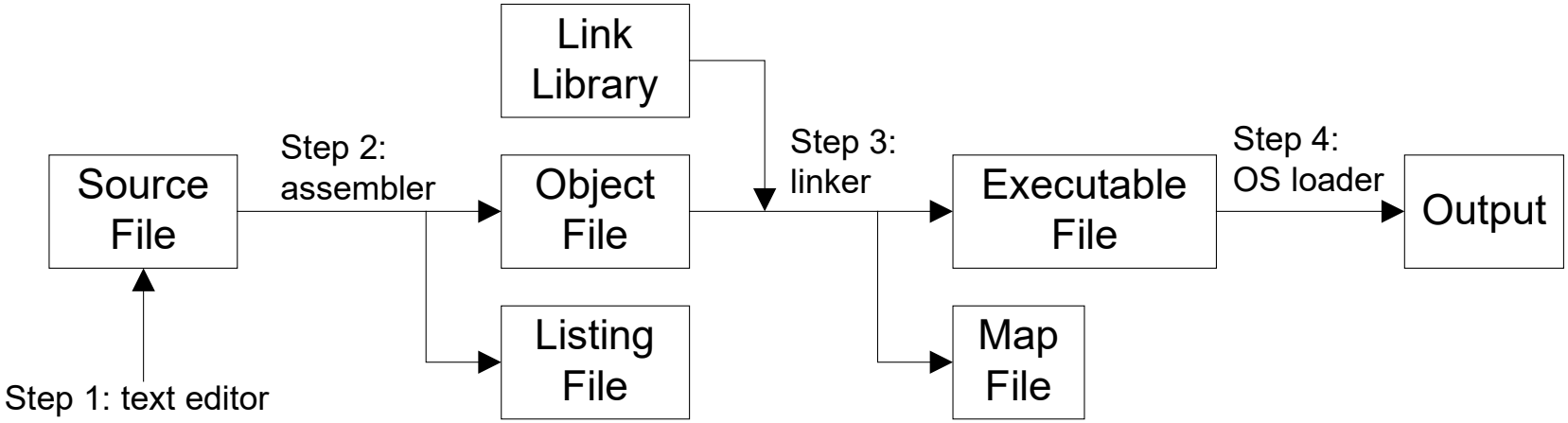


Figure: Memory read cycle.

How Program Run?

Assembling, Linking, and Running Programs



- The diagram describes the steps from creating a source program through executing the compiled program.
- If the source code is modified, Steps 2 through 4 must be repeated.

Listing File

- Use it to see how your program is compiled.
- Contains:
  - *source code*
  - *addresses*
  - *object code (machine language)*
  - *segment names*
  - *symbols (variables, procedures, and constants)*
- **Example**: `addSub.lst`

```
Microsoft (R) Macro Assembler Version 9.00.30729.01    05/07/09
16:43:07
Add and Subtract        (AddSub.asm)         Page 1 - 1

TITLE Add and Subtract                  (AddSub.asm)

; This program adds and subtracts 32-bit integers.

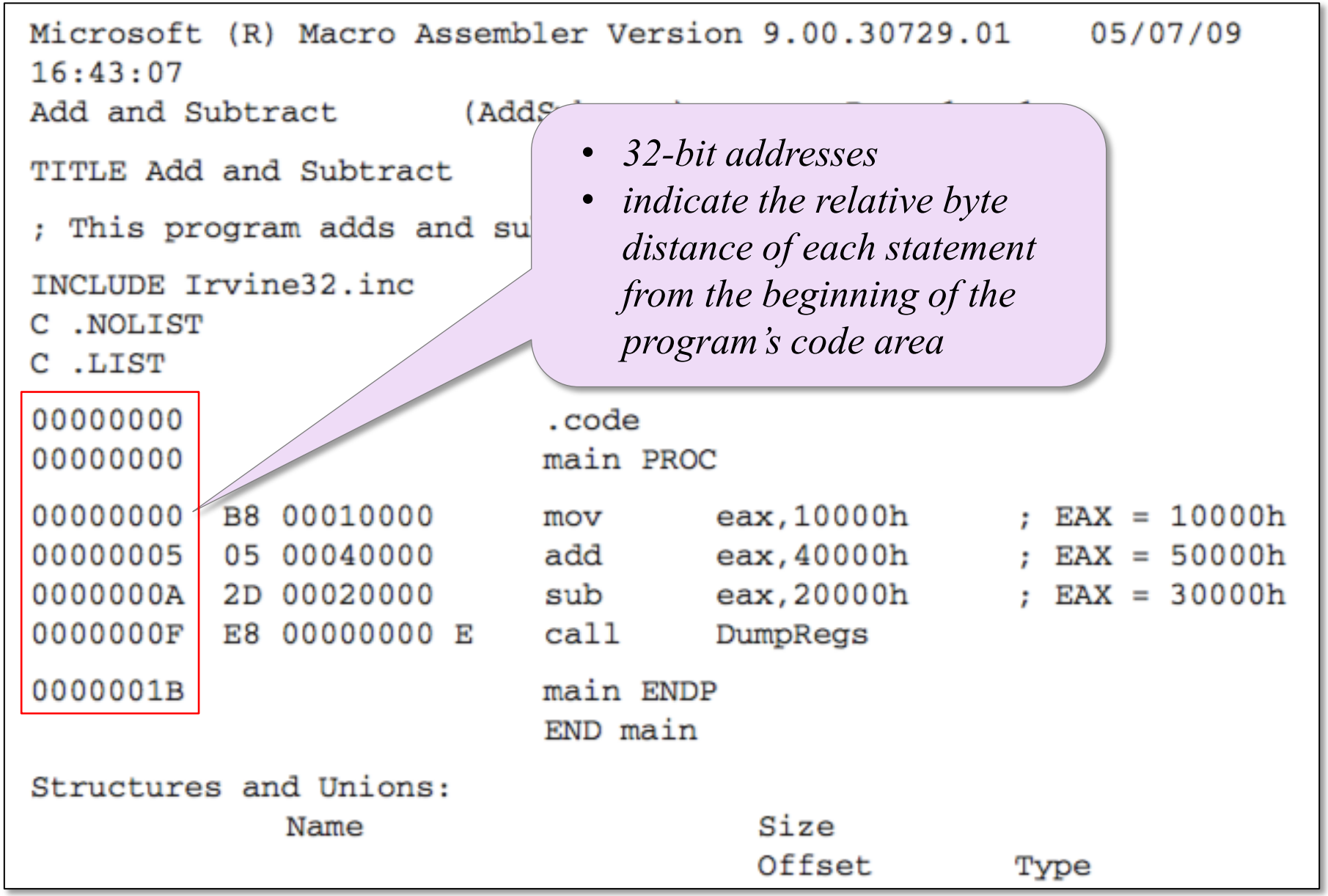
INCLUDE Irvine32.inc
C .NOLIST
C .LIST

00000000                        .code
00000000                        main PROC

00000000  B8 00010000           mov     eax,10000h      ; EAX = 10000h
00000005  05 00040000           add     eax,40000h      ; EAX = 50000h
0000000A  2D 00020000           sub     eax,20000h      ; EAX = 30000h
0000000F  E8 00000000 E         call    DumpRegs

0000001B                        main ENDP
                                END main

Structures and Unions:
                Name                            Size
                                                Offset      Type
```


Microsoft (R) Macro Assembler Version 9.00.30729.01 05/07/09
16:43:07
Add and Subtract (AddS
TITLE Add and Subtract
; This program adds and su
INCLUDE Irvine32.inc
C .NOLIST
C .LIST
00000000 .code
00000000 main PROC
00000000 B8 00010000 mov eax,10000h ; EAX = 10000h
00000005 05 00040000 add eax,40000h ; EAX = 50000h
0000000A 2D 00020000 sub eax,20000h ; EAX = 30000h
0000000F E8 00000000 E call DumpRegs
0000001B main ENDP
END main
Structures and Unions:
Name
Size
Offset
Type
• 32-bit addresses
• indicate the relative byte distance of each statement from the beginning of the program's code area

- *contain no executable instructions*
- *... directives,*

```
Microsoft (R) Macro Assembler Versi
16:43:07
Add and Subtract            (AddSub.asm)

TITLE Add and Subtract                     (Add          m)

; This program adds and subtracts 32-bi       tegers.

INCLUDE Irvine32.inc
C .NOLIST
C .LIST

00000000                    .code
00000000                    main PROC

00000000  B8 00010000       mov     eax,10000h      ; EAX = 10000h
00000005  05 00040000       add     eax,40000h      ; EAX = 50000h
0000000A  2D 00020000       sub     eax,20000h      ; EAX = 30000h
0000000F  E8 00000000 E     call    DumpRegs

0000001B                    main ENDP
                            END main

Structures and Unions:
                Name                        Size
                                            Offset      Type
```

- *machine language instructions, each 5 bytes long*
- *represented as the hexadecimal values in the second column, such as* B8 00010000.

```
Microsoft (R) Mac                                              05/07/09
16:43:07
Add and Subtract

TITLE Add and Sub

; This program add

INCLUDE Irvine32.inc
C .NOLIST
C .LIST

00000000                        .code
00000000                        main PROC

00000000  B8 00010000           mov     eax,10000h      ; EAX = 10000h
00000005  05 00040000           add     eax,40000h      ; EAX = 50000h
0000000A  2D 00020000           sub     eax,20000h      ; EAX = 30000h
0000000F  E8 00000000 E         call    DumpRegs

0000001B                        main ENDP
                                END main

Structures and Unions:
                Name                            Size
                                                Offset      Type
```

Map File

- Information about each program segment:
  - *starting address*
  - *ending address*
  - *size*
  - *segment type*

- **Example**: `addSub.map`

AddSubMap.txt

```
 Start  Stop   Length Name                   Class
 00000H 006E2H 006E3H _TEXT                  CODE
 006E4H 008FDH 0021AH _DATA                  DATA
 00900H 028FFH 02000H STACK                  STACK
 02900H 02AFFH 00200H _BSS                   BSS

 Origin   Group
 006E:0   DGROUP

Program entry point at 0000:0000
```

Module 4
Instruction Set Architecture (ISA)

4.1 Introduction

4.2 Machine Instruction Characteristics

4.3 Types of Operands

4.4 Addressing Modes

4.5 Instruction Formats

4.6 Summary

- Elements of a Machine Instruction
- Instruction Representation
- Instruction Types
- Number of Addresses
- Instruction Set Design

Elements of a Machine Instruction

4

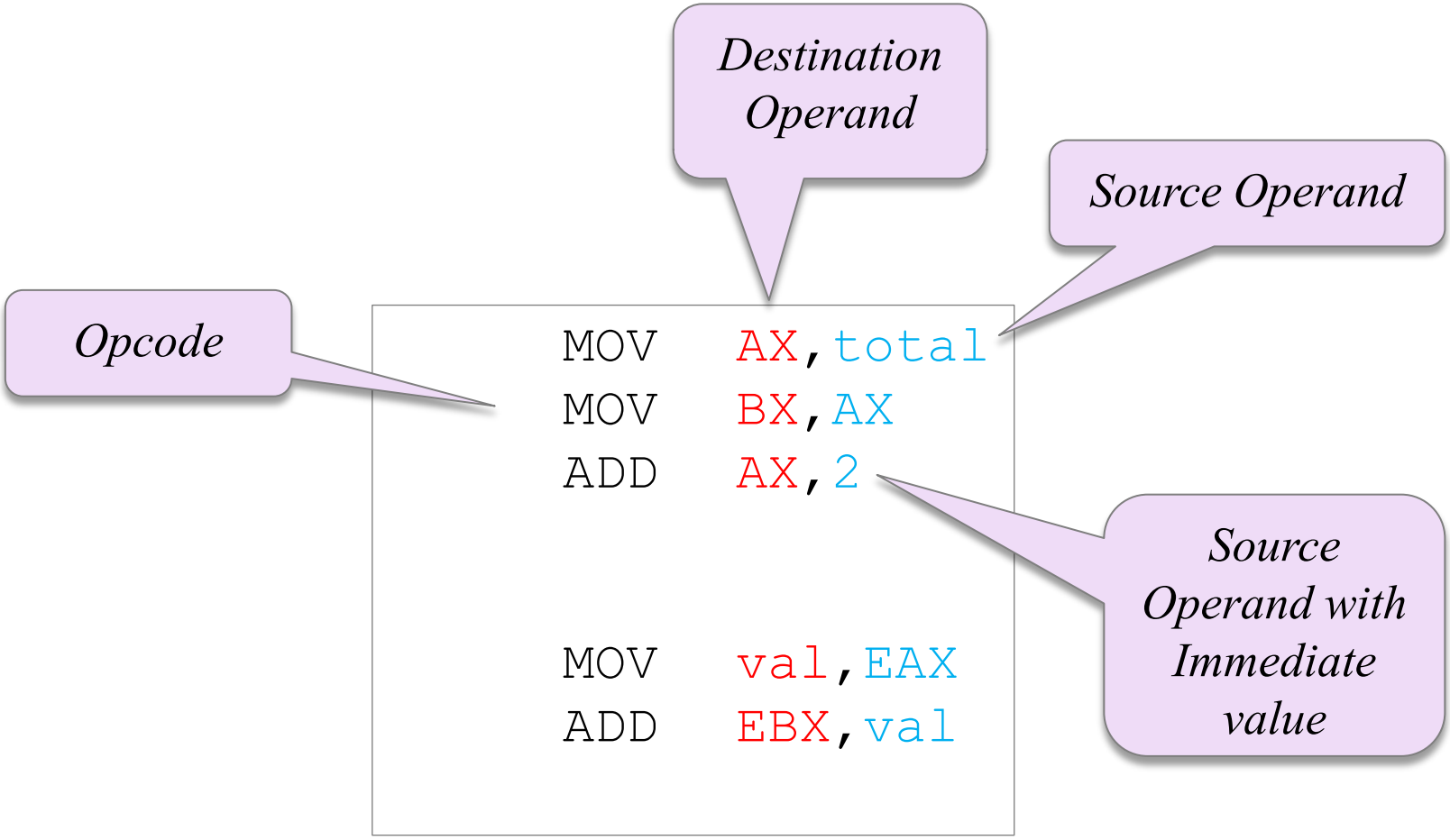
- The operation of the processor is determined by the instructions it executes, referred to as machine instructions or computer instructions.
- The collection of different instructions that the processor can execute is referred to as the processor's instruction set.
- Each instruction must contain the information required by the processor for execution.

Table: The elements of a machine instruction.

Elements	Description
Operation code *(Opcode)*	○ Specifies the operation to be performed, represented as a binary code (e.g., `MOV`, `ADD`, `SUB`).
Source operand reference	○ The operation may involve one or more source operands, that is, operands that are inputs for the operation.
Result operand reference *(Destination)*.	○ The operation may produce a result.
Next instruction reference	○ This tells the processor where to fetch the next instruction after the execution of this instruction is completed.

Invisible

Example 1: Portion of an assembly language.

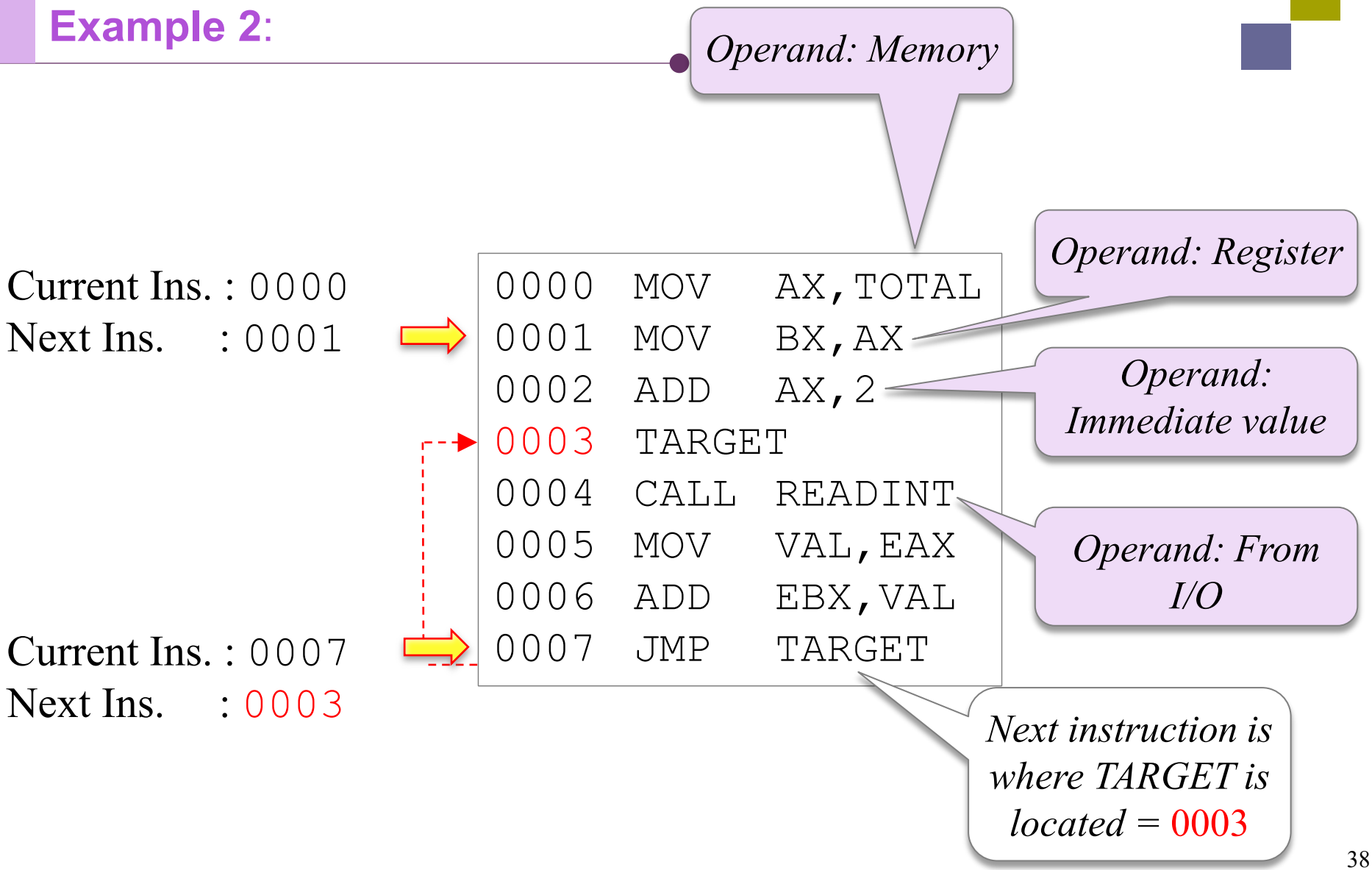


- *Source* and result operands (*Destination*) can be in one of four areas:

Section 4.4: Addressing Mode

  - Main or virtual memory : Memory address for both must be supplied.
  - Processor (CPU) registers : One or more registers that can be referenced by instructions.
  - Immediate : The value of the operand is contained in the field in the instruction executed.
  - I/O device – Instruction specifies the I/O module and device for the operation

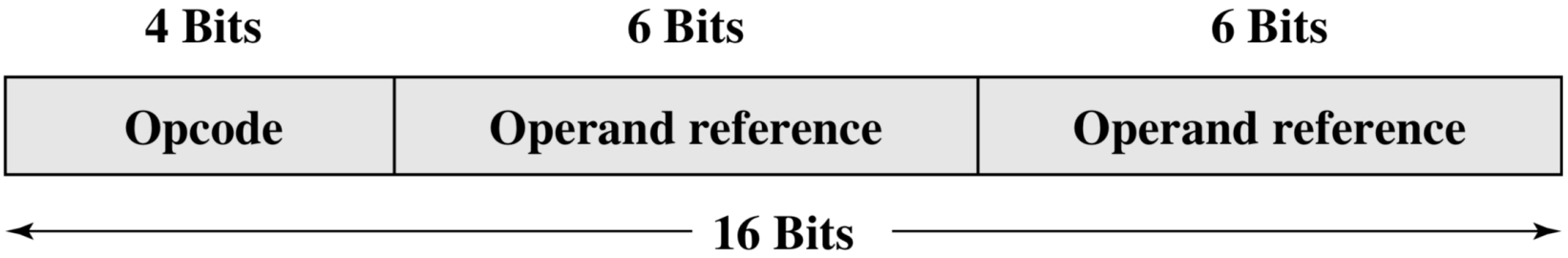
Example 2:


Operand: Memory
Operand: Register
Operand: Immediate value
Operand: From I/O
Next instruction is where TARGET is located = 0003
Current Ins. : 0000
Next Ins. : 0001
Current Ins. : 0007
Next Ins. : 0003
0000 MOV AX,TOTAL
0001 MOV BX,AX
0002 ADD AX,2
0003 TARGET
0004 CALL READINT
0005 MOV VAL,EAX
0006 ADD EBX,VAL
0007 JMP TARGET

4 Instruction Representation

- Each instruction is represented by a sequence of bits that divided into fields, corresponding to the constituent elements of the instruction.

- **Example** of simple instruction format:



- **Example**:

Instruction type	Opcode	Symbolic representation	Description
Data transfer	00001010	LOAD MQ,M(X)	Transfer contents of memory location X to register MQ

What the processor (CPU) see.

What the programmer see.

- During instruction execution:
  - an instruction is read into an Instruction Register (IR) in the processor.
  - The processor must be able to extract the data from the various instruction fields to perform the required operation.

A single C instruction may require 3 machine instructions; This is typical of the relationship between a high-level language and a machine language.

4

- Consider a high-level language instruction that could be expressed in a language such as C.
- **Example**:

```
Total = Total + stuff
```

- In assembly language:

1) Load a register with the contents of memory (for `Total`).
2) Add the contents of memory (for `stuff`) to the register.
3) Store the content of the register to memory location (for `Total`).

- Translating the language:

English: `Total` is assigned the sum of `Total` and `stuff`.

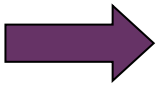


High-Level Language: `Total = Total + stuff`



A statement in a high-level language is translated typically into several machine-level instructions

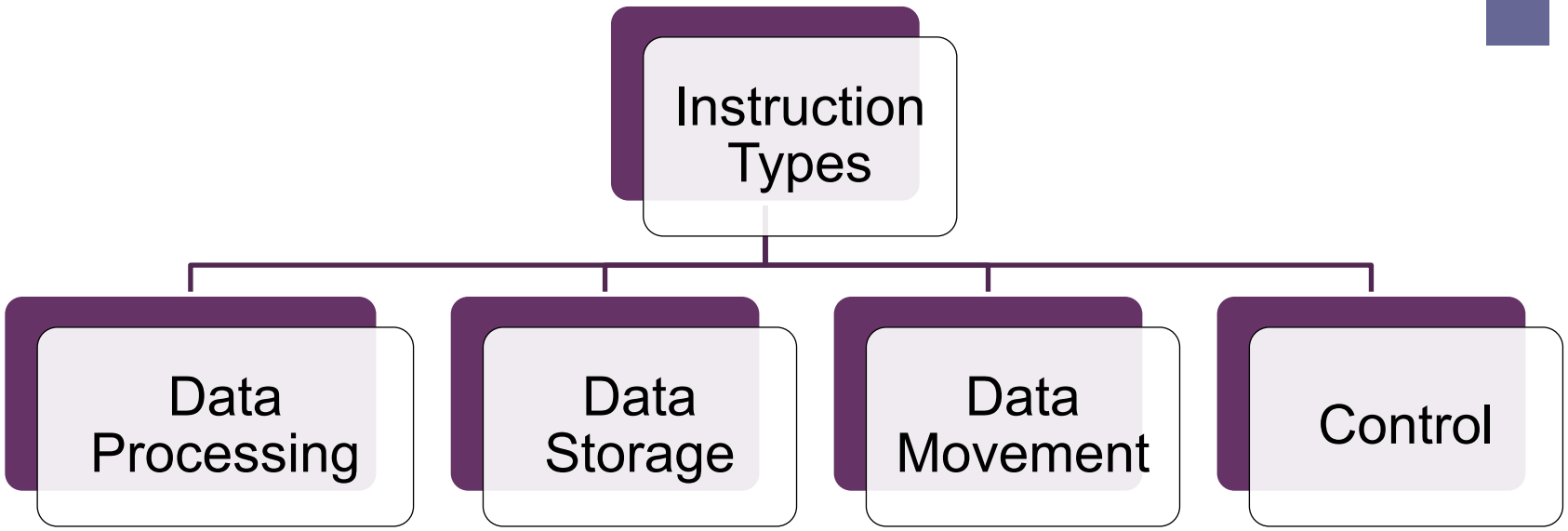
```
mov  eax,Total
add  eax,stuff
mov  Total,eax
```



```
A1 00404000
83 00404004
A3 00404000
```

Intel Assembly Language:

Intel Machine Code:



- *Arithmetic and logic instructions.*
- *Movement of data in/out register and/or memory.*
- *I/O instructions.*
- *Test and branch instructions.*

Figure: Categories of instruction types.

Table: Examples of *data processing*.

Instruction	Description
Arithmetic	
ADD	Add operands.
SUB	Subtract operands.
MUL	Unsigned integer multiplication, with byte, word, or double word operands, and word, doubleword, or quadword result.
IDIV	Signed divide.
Logical	
AND	AND operands.
BTS	Bit test and set. Operates on a bit field operand. The instruction copies the current value of a bit to flag CF and sets the original bit to 1.
BSF	Bit scan forward. Scans a word or doubleword for a 1-bit and stores the number of the first 1-bit into a register.
SHL/SHR	Shift logical left or right.
SAL/SAR	Shift arithmetic left or right.

Table: Examples of *data storage* and *data movement*.

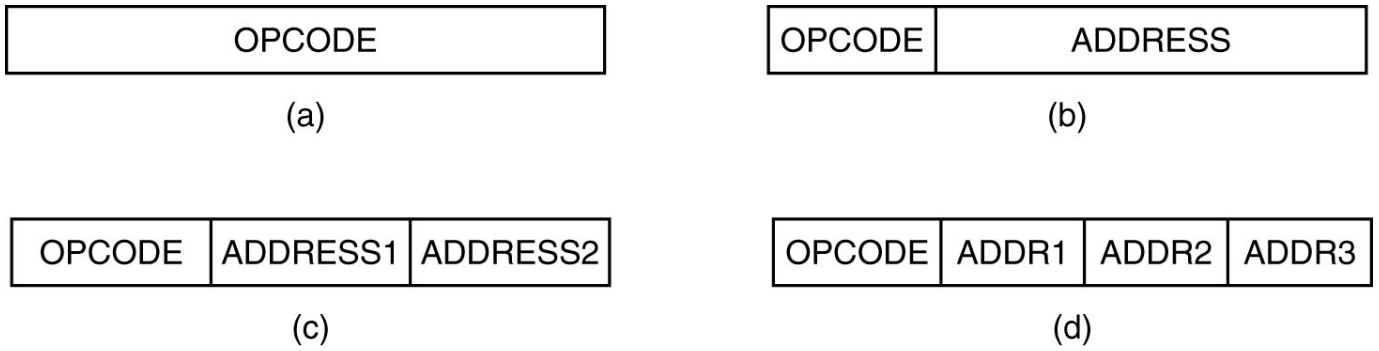
Instruction	Description
Data Movement	
MOV	Move operand, between registers or between register and memory.
PUSH	Push operand onto stack.
PUSHA	Push all registers on stack.
MOVSX	Move byte, word, dword, sign extended. Moves a byte to a word or a word to a doubleword with twos-complement sign extension.
LEA	Load effective address. Loads the offset of the source operand, rather than its value to the destination operand.
XLAT	Table lookup translation. Replaces a byte in AL with a byte from a user-coded translation table. When XLAT is executed, AL should have an unsigned index to the table. XLAT changes the contents of AL from the table index to the table entry.
IN, OUT	Input, output operand from I/O space.

Table: Examples of *control* (test and branch).

Instruction	Description
Control Transfer	
JMP	Unconditional jump.
CALL	Transfer control to another location. Before transfer, the address of the instruction following the CALL is placed on the stack.
JE/JZ	Jump if equal/zero.
LOOPE/LOOPZ	Loops if equal/zero. This is a conditional jump using a value stored in register ECX. The instruction first decrements ECX before testing ECX for the branch condition.
INT/INTO	Interrupt/Interrupt if overflow. Transfer control to an interrupt service routine.

4 Number of Addresses

- One of the traditional ways in describing processor architecture is using the number of addresses contained in each instruction .
- Number of addresses refers to *how many operands can an instruction take*.



Four common instruction formats:

(a) Zero-address instruction. (b) One-address instruction

(c) Two-address instruction. (d) Three-address instruction.

Programs to Execute $Y = \dfrac{A - B}{C + (D \times E)}$

Instruction		Comment
SUB	Y, A, B	Y ← A – B
MPY	T, D, E	T ← D × E
ADD	T, T, C	T ← T + C
DIV	Y, Y, T	Y ← Y ÷ T

(a) Three-address instructions

3 addresses → 4 instructions

Programs to Execute $Y = \dfrac{A - B}{C + (D \times E)}$

Instruction		Comment
MOVE	Y, A	Y ← A
SUB	Y, B	Y ← Y – B
MOVE	T, D	T ← D
MPY	T, E	T ← T × E
ADD	T, C	T ← T + C
DIV	Y, T	Y ← Y ÷ T

(b) Two-address instructions

2 addresses → 6 instructions

Programs to Execute $Y = \dfrac{A - B}{C + (D \times E)}$

Instruction		Comment
LOAD	D	AC ← D
MPY	E	AC ← AC × E
ADD	C	AC ← AC + C
STOR	Y	Y ← AC
LOAD	A	AC ← A
SUB	B	AC ← AC – B
DIV	Y	AC ← AC ÷ Y
STOR	Y	Y ← AC

(c) One-address instructions

1 address → 8 instructions

Programs to Execute $Y = \dfrac{A - B}{C + (D \times E)}$

```
PUSH C
PUSH D
PUSH E
MUL
ADD
PUSH B
PUSH A
SUB
DIV
POP Y
```

(d) Zero-address instructions

0 address → 10 instructions

Programs to Execute $Y = \frac{A - B}{C + (D \times E)}$

Instruction		Comment
MOVE	Y, A	Y ← A
SUB	Y, B	Y ← Y – B
MOVE	T, D	T ← D
MPY	T, E	T ← T × E
ADD	T, C	T ← T + C
DIV	Y, T	Y ← Y ÷ T

2 addresses → 6 instructions

(b) Two-address instructions

Instruction		Comment
SUB	Y, A, B	Y ← A – B
MPY	T, D, E	T ← D × E
ADD	T, T, C	T ← T + C
DIV	Y, Y, T	Y ← Y ÷ T

3 addresses → 4 instructions

(a) Three-address instructions

Instruction		Comment
LOAD	D	AC ← D
MPY	E	AC ← AC × E
ADD	C	AC ← AC + C
STOR	Y	Y ← AC
LOAD	A	AC ← A
SUB	B	AC ← AC – B
DIV	Y	AC ← AC ÷ Y
STOR	Y	Y ← AC

1 address → 8 instructions

(c) One-address instructions

0 address → 10 instructions

```
PUSH C
PUSH D
PUSH E
MUL
ADD
PUSH B
PUSH A
SUB
DIV
POP Y
```

(a) Three-address instructions

$$Y = \frac{A - B}{C + (D \times E)}$$

Instruction		Comment
SUB	Y, A, B	Y ← A − B
MPY	T, D, E	T ← D × E
ADD	T, T, C	T ← T + C
DIV	Y, Y, T	Y ← Y ÷ T

4 instructions

- 3 addresses: *Operand 1*, *Operand 2*, *Result* (Destination).
- May has a fourth address - next instruction (usually implicit, obtained from PC (*Program Counter*)).
- Example above: *T* = temporary location used to store intermediate results.
- Not common in use.
- Needs very long words to hold everything.

(b) Two-address instructions

$$Y = \frac{A - B}{C + (D \times E)}$$

Instruction	Comment
MOVE Y, A	Y ← A
SUB Y, B	Y ← Y − B
MOVE T, D	T ← D
MPY T, E	T ← T × E
ADD T, C	T ← T + C
DIV Y, T	Y ← Y ÷ T

6 instructions

- 2 addresses: *Operand*, *Result* (Destination).
- Reduces length of instruction and space requirements.
- Requires some extra works:
  - Temporary storage to hold some results.
  - Done to avoid altering the operand value.

(c) One-address instructions

$$Y = \frac{A - B}{C + (D \times E)}$$

Instruction	Comment
LOAD D	AC ← D
MPY E	AC ← AC × E
ADD C	AC ← AC + C
STOR Y	Y ← AC
LOAD A	AC ← A
SUB B	AC ← AC − B
DIV Y	AC ← AC ÷ Y
STOR Y	Y ← AC

8 instructions

- 1 address
- Implicit second address.
- Usually use a *CPU register* (accumulator)
  - Supplies 1 operand and store result.
  - One memory address used for another operand.
- Common on early machines.

(d) Zero-address instructions

$$Y = \frac{A - B}{C + (D \times E)}$$

```
PUSH   C
PUSH   D
PUSH   E
MUL
ADD
PUSH   B
PUSH   A
SUB
DIV
POP    Y
```

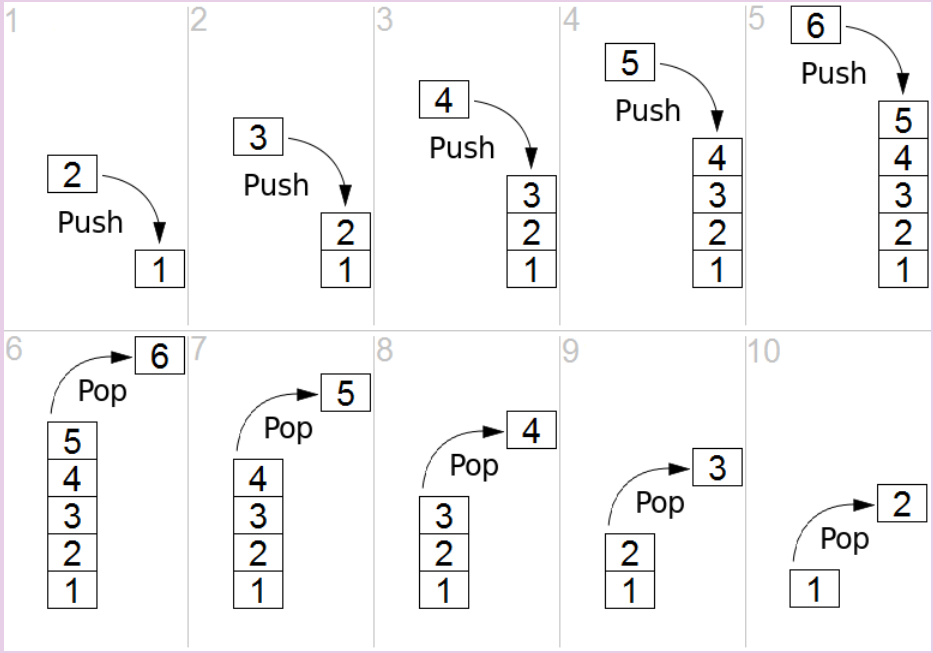
10 instructions

- 0 (zero) address
- All addresses implicit.
- Usually use a *stack* (a push down stack in CPU).
- There are two *Opcodes* with one *operand*:

  • **PUSH op**, **POP op**

(d) Zero-address instructions

Stack Machine



- A *stack* is an abstract data type and data structure based on the principle of *Last In First Out* (LIFO).
- *Stack machine*: Java Virtual Machine.
- *Call stack* of a program, also known as a function stack, execution stack, control stack, or simply the stack.
- Application: *Reverse Polish Notation (RPN)*, *Depth-First-Search (DFS)*

RPN will be discussed in next example

https://en.wikipedia.org/wiki/Stack_(abstract_data_type)

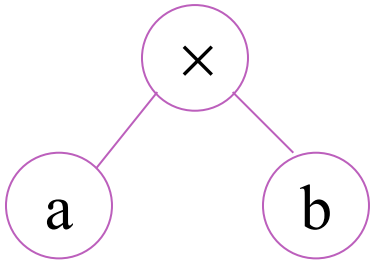
Stack Machine

- In computer engineering and in programming language implementations, a stack machine is a real or emulated computer that uses a pushdown stack rather than individual machine registers *to evaluate each sub-expression in the program*.
- A stack computer is programmed with a *reverse Polish notation instruction set*.

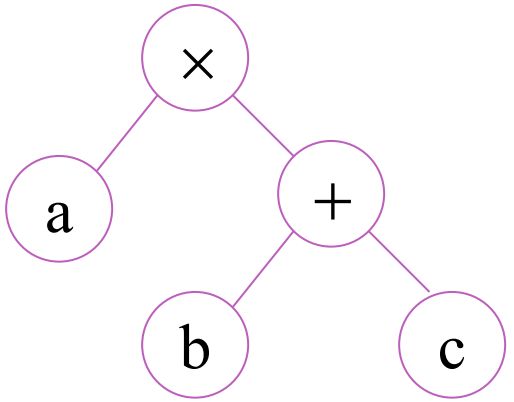
source: Wikipedia

Expression tree

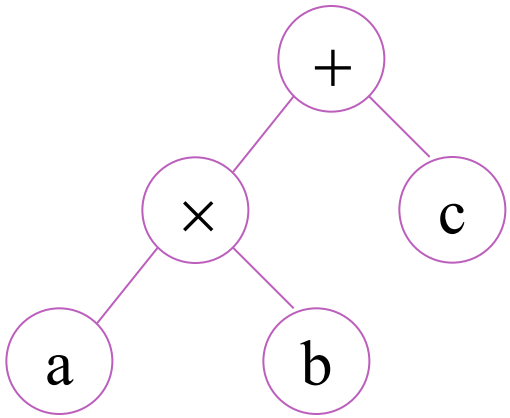
$a \times b$



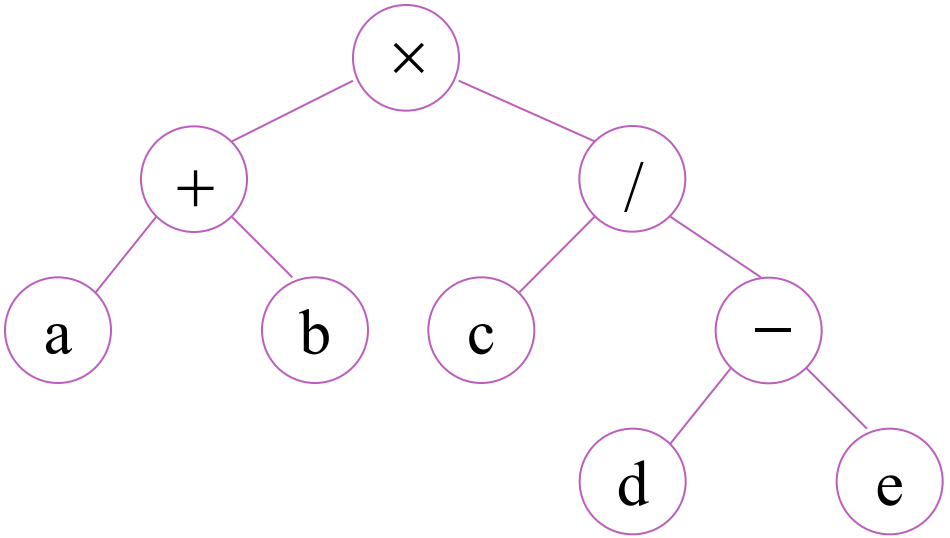
$a \times (b + c)$



$a \times b + c$



$(a + b) \times (c / (d - e))$



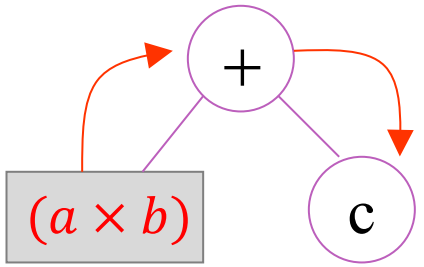
Expression tree:

(a) Infix notation:

Left-Parent-Right order

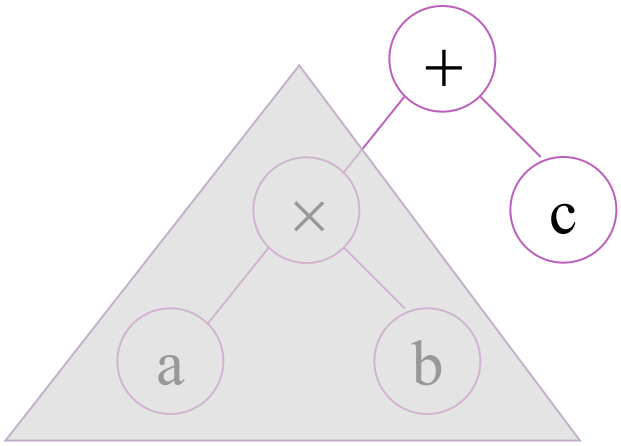
Recursive Left-Parent-Right again

infix: $(a \times b) + c$



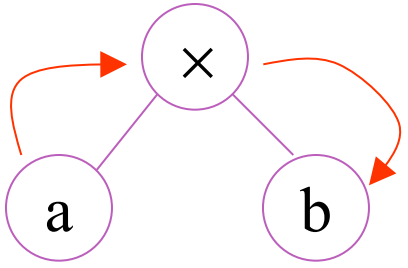
Replace subtree with infix notation

$a \times b + c$



Left child of root "+"

Recursive Left-Parent-Right



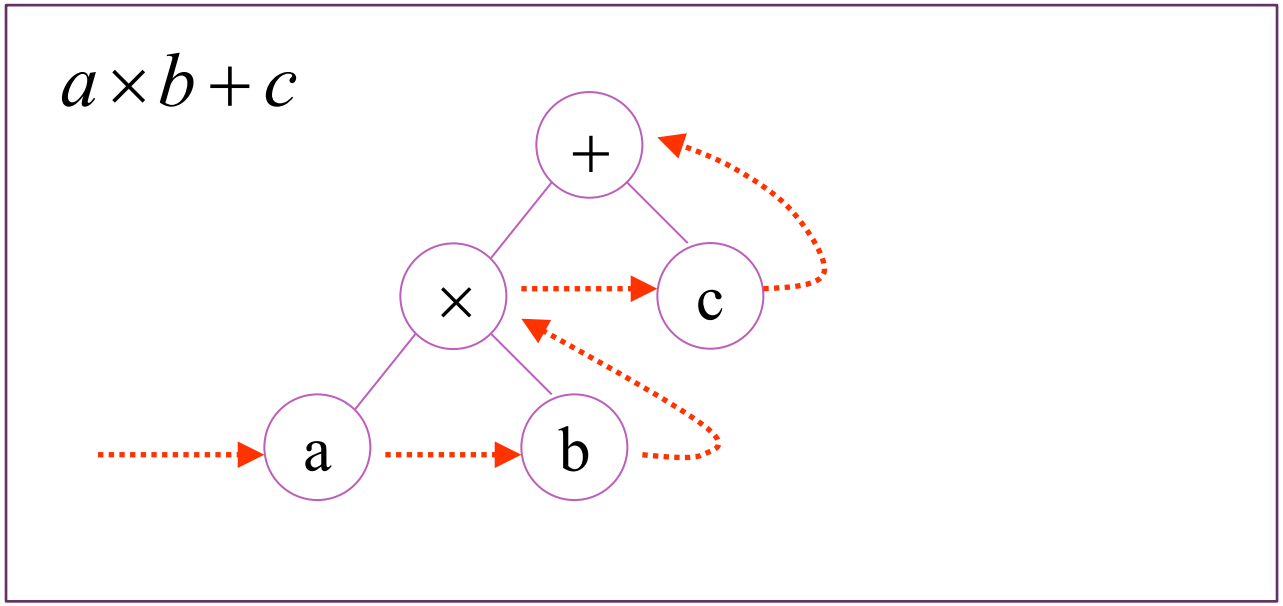
infix: $(a \times b)$

Expression tree:

(b) Postfix notation:

Left-Right-Parent order

$postfix: ab \times c +$



Expression tree:

$$a \times b + c$$

(a) Infix notation:

infix: $(a \times b) + c$

(b) Postfix notation:

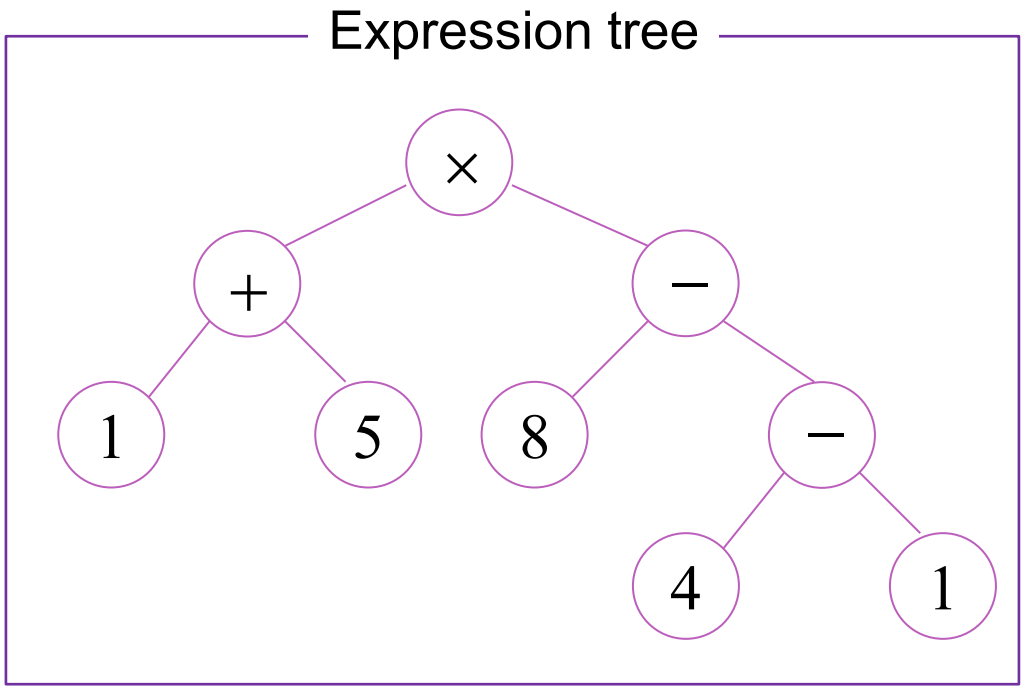
$postfix$: $ab \times c +$

Reverse Polish Notation (RPN)

- Precedence of multiplication is higher than addition, we need parenthesis to guarantee execution order as for infix notation.
- However in the early 1950s, the Polish logician Jan Lukasiewicz observed that parentheses are not necessary in postfix notation, called *RPN* (*Reverse Polish Notation*).
- The Reverse Polish scheme was proposed by F.L. Bauer and E.W. Dijkstra in the early 1960s to reduce computer memory access and utilize the *stack* to evaluate expressions.

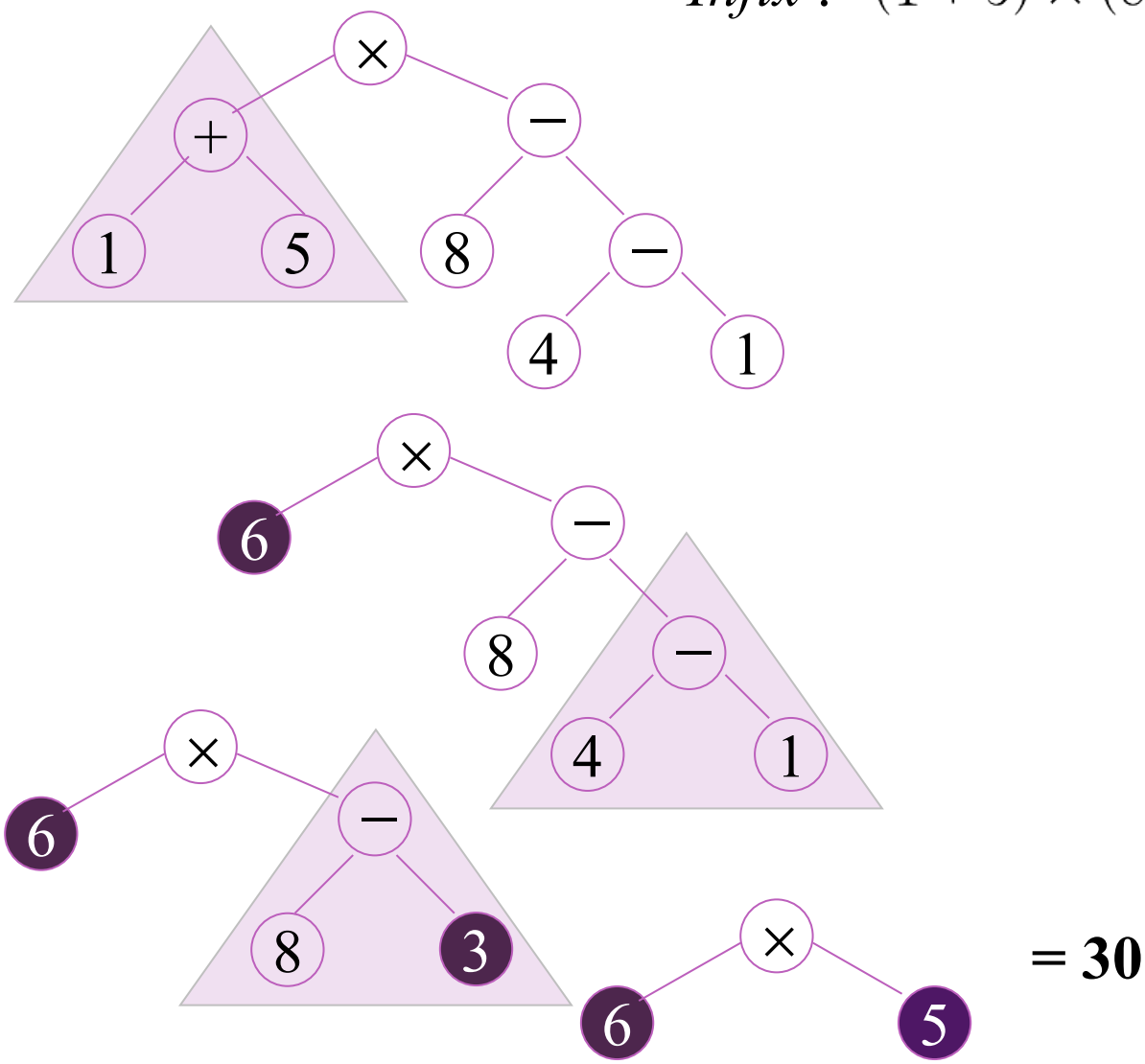
Example: Reverse Polish Notation (RPN) → *Postfix order*

$$Infix : (1+5) \times (8-(4-1))$$



Example: Reverse Polish Notation (RPN) → *Postfix order* [1]

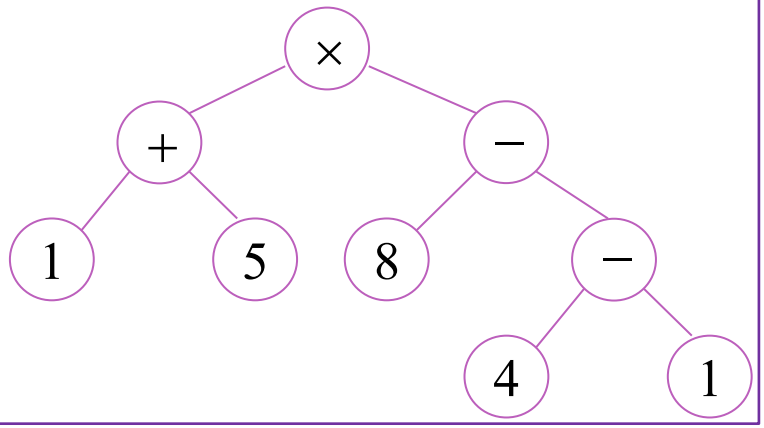
Infix : $(1+5)\times(8-(4-1))$



Example: Reverse Polish Notation (RPN) → *Postfix order* [1]

Infix : $(1+5)\times(8-(4-1))$

Expression tree



Postfix : $15+841--\times$

$1+5=6$

$6841--\times$

$4-1=3$

$683-\times$

$8-3=5$

$65\times$

$6\times5=30$

30

Example: Evaluate RPN expression [1]

Expression

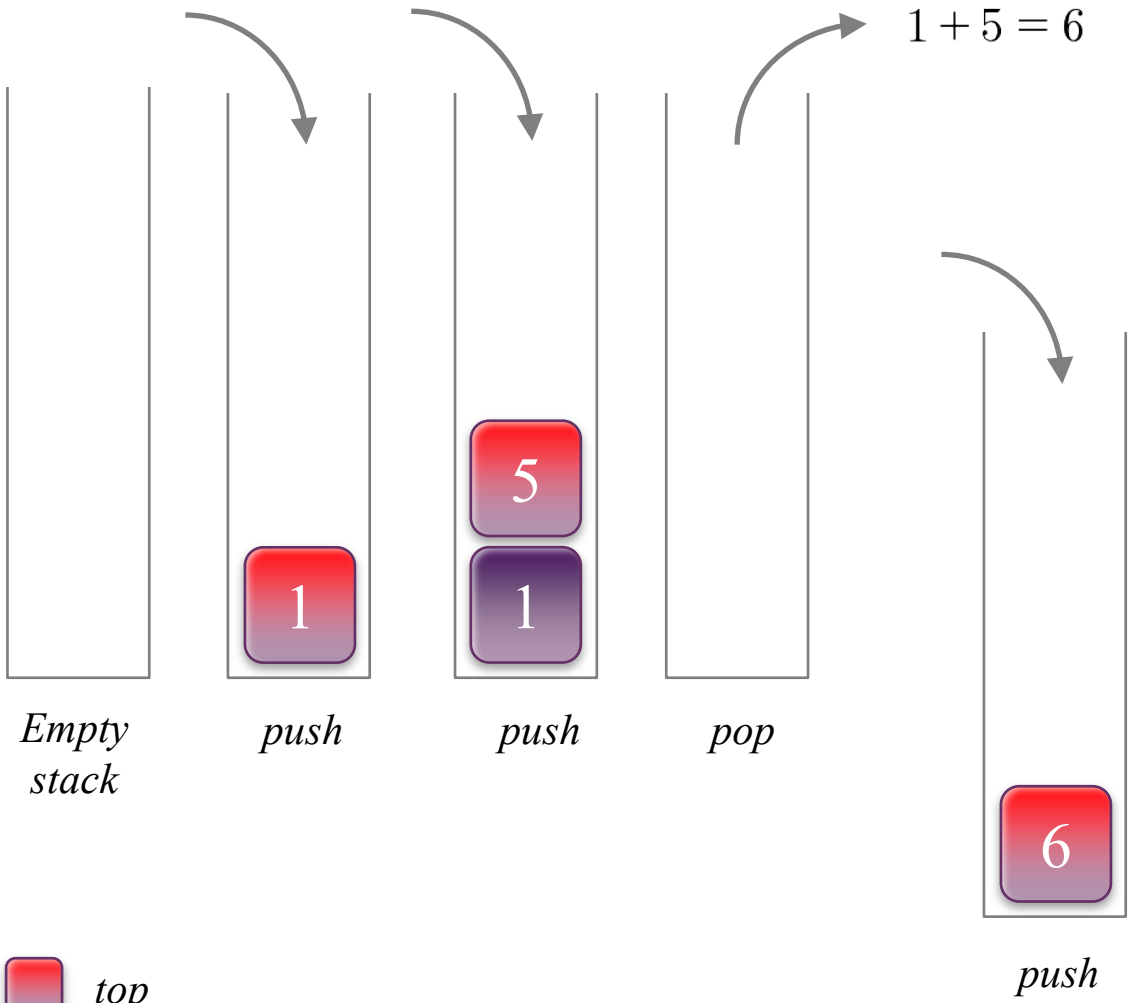
30

Infix : $(1+5)\times(8-(4-1))$

Postfix : $15+841--\times$

- **Scanned** from left to right until an operator is found, then the last two operands must be retrieved and combined.
- Order of operands satisfy LIFO, so we can use **stack** to store operands and then evaluate RPN expression.

Example: Evaluate RPN expression [2]



$15 + 841 - -\times$

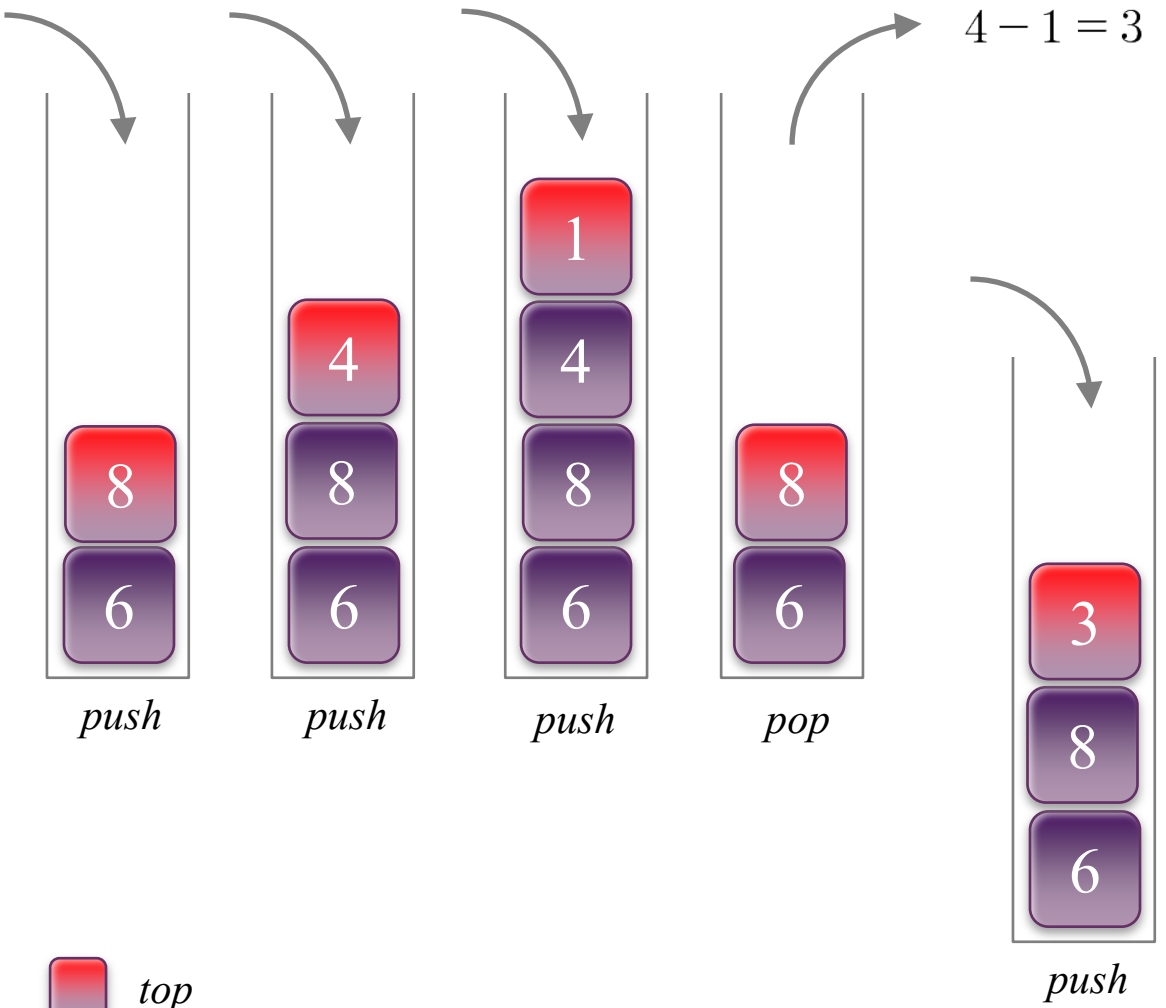
$6841 - -\times$

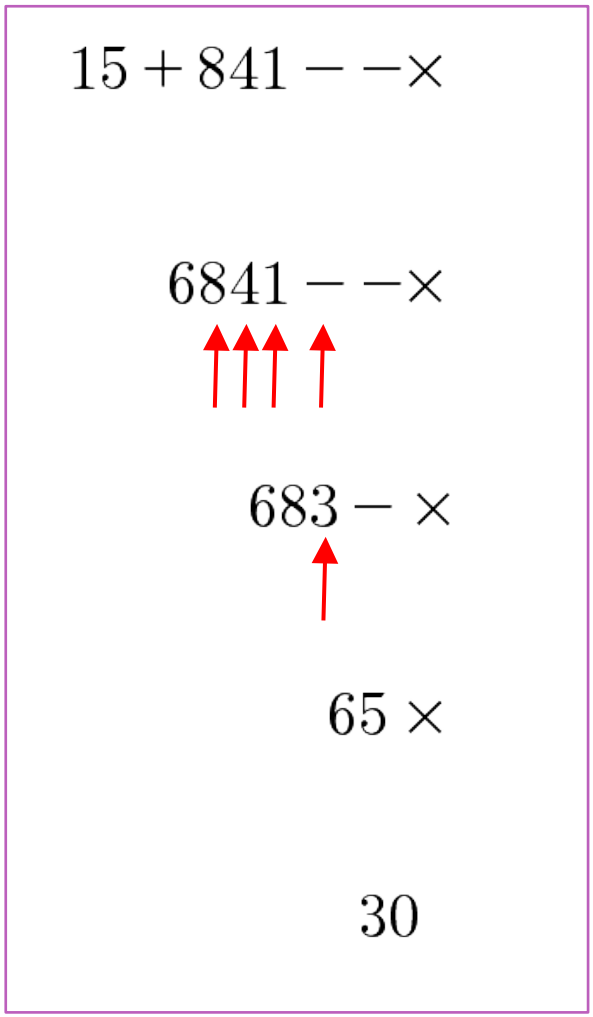
$683 - \times$

$65 \times$

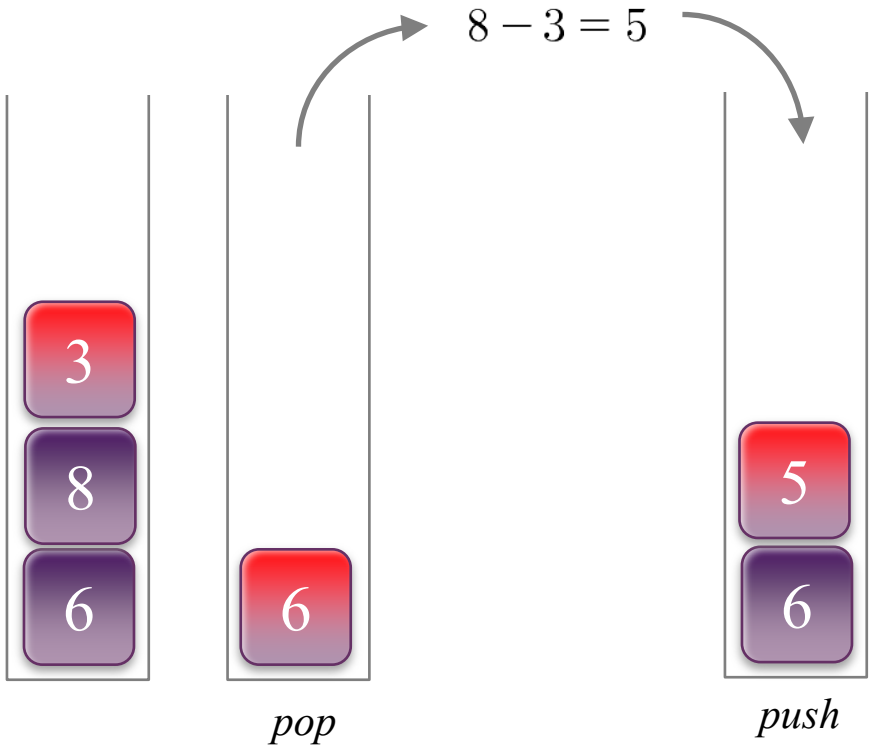
30

Example: Evaluate RPN expression [2]





Example: Evaluate RPN expression [2]



$15 + 841 - - \times$

$6841 - - \times$

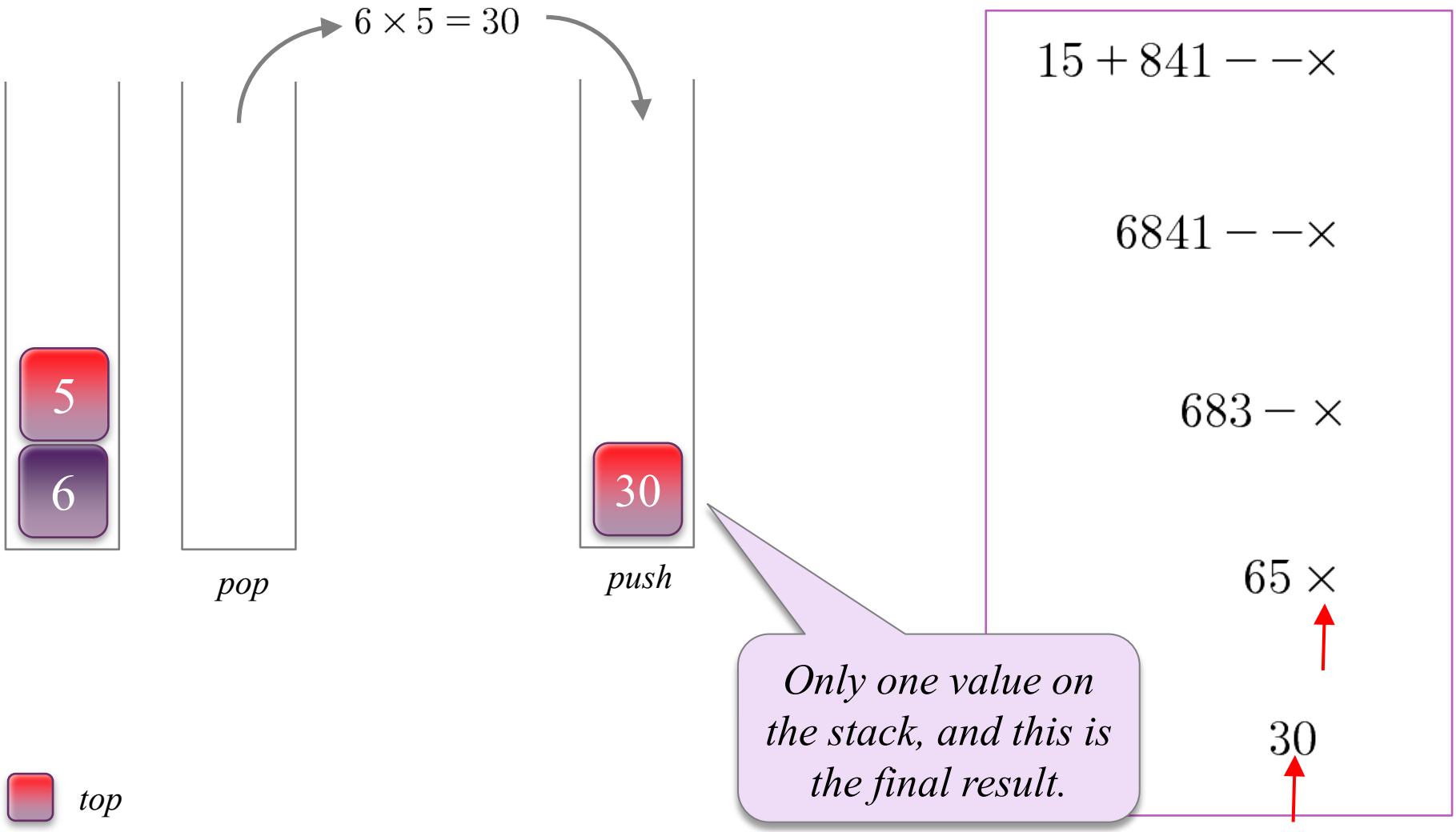
$683 - \times$

$65 \times$

30

top

Example: Evaluate RPN expression [2]



How Many Addresses?

- Number of addresses per instruction is a basic design decision.

- More addresses:
  - More complex (powerful?) instructions.
  - Fewer instructions per program.
  - More registers:

    → Inter-register operations are quicker.

- Fewer addresses:
  - Less complex (powerful?) instructions.
  - More instructions per program.
  - Faster fetch/execution of instructions.

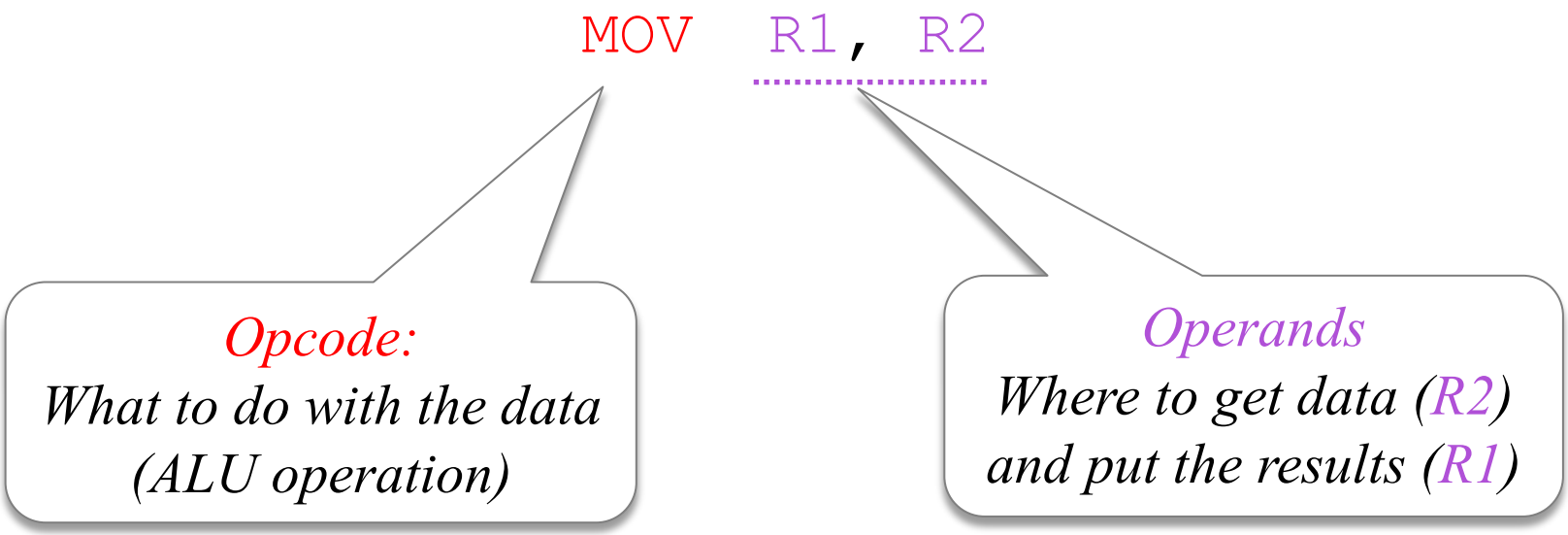
Module 4
Instruction Set Architecture (ISA)

4.1 Introduction
4.2 Machine Instruction Characteristics
4.3 Types of Operands
4.4 Addressing Modes
4.5 Instruction Formats
4.6 Summary

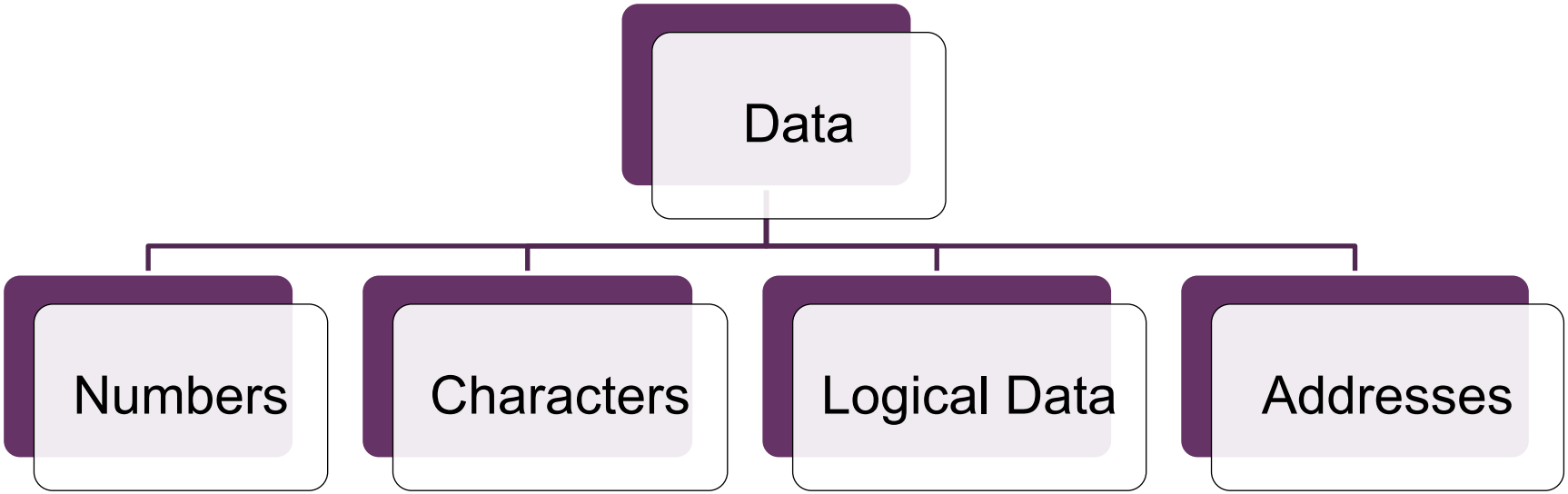
- Overview
- Numbers
- Characters
- Logical Data

Overview

- Assembly language built from two pieces:



- Machine instructions operate on data.
- The general categories of data:



(Section 4.4)

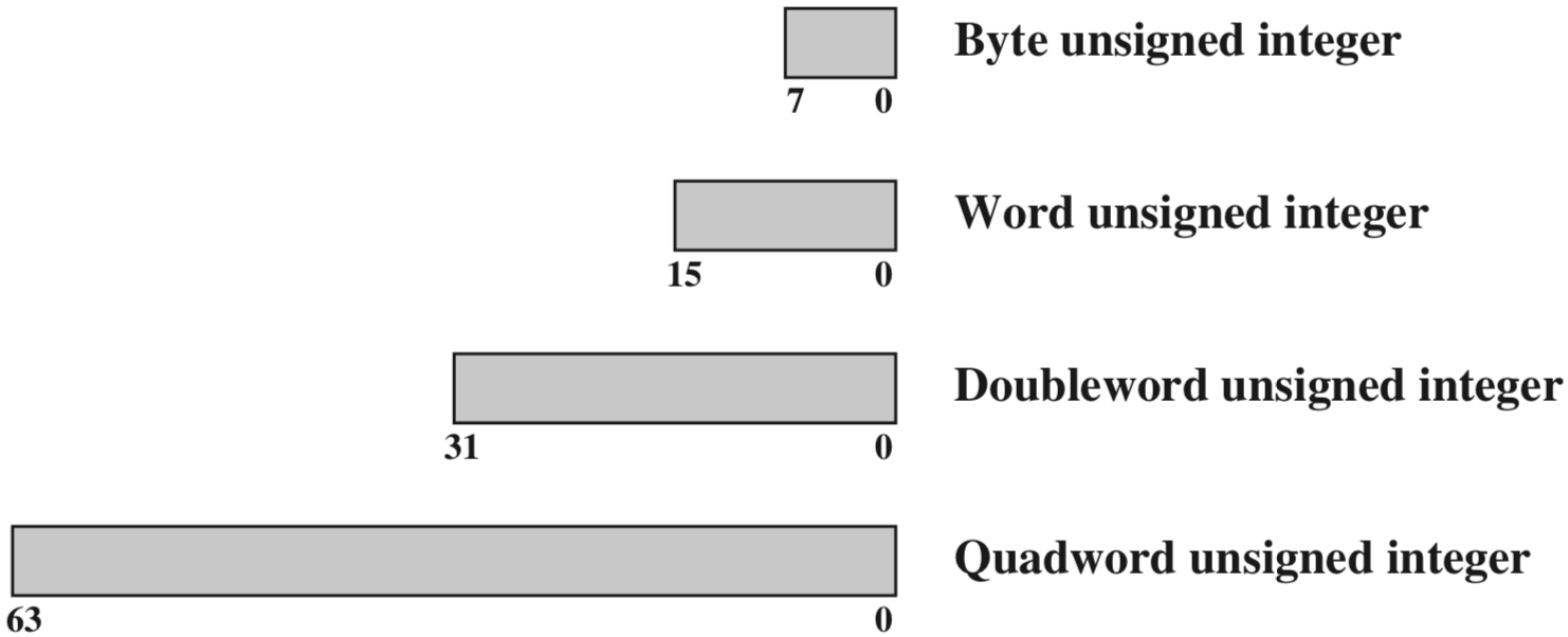
4 Numbers

- All machine languages include numeric data types.
- An important distinction between numbers used in ordinary mathematics and numbers stored in a computer is that the latter are limited: (2 reasons)
  - There is a limit to the magnitude of numbers representable on a machine.
  - A limit to the precision of floating-point numbers.
- 3 types of numerical data:
  - Binary integer or binary fixed point
  - Binary floating point
  - Decimal

Example 3a: x86 numerical data types.

(Unsigned integers)

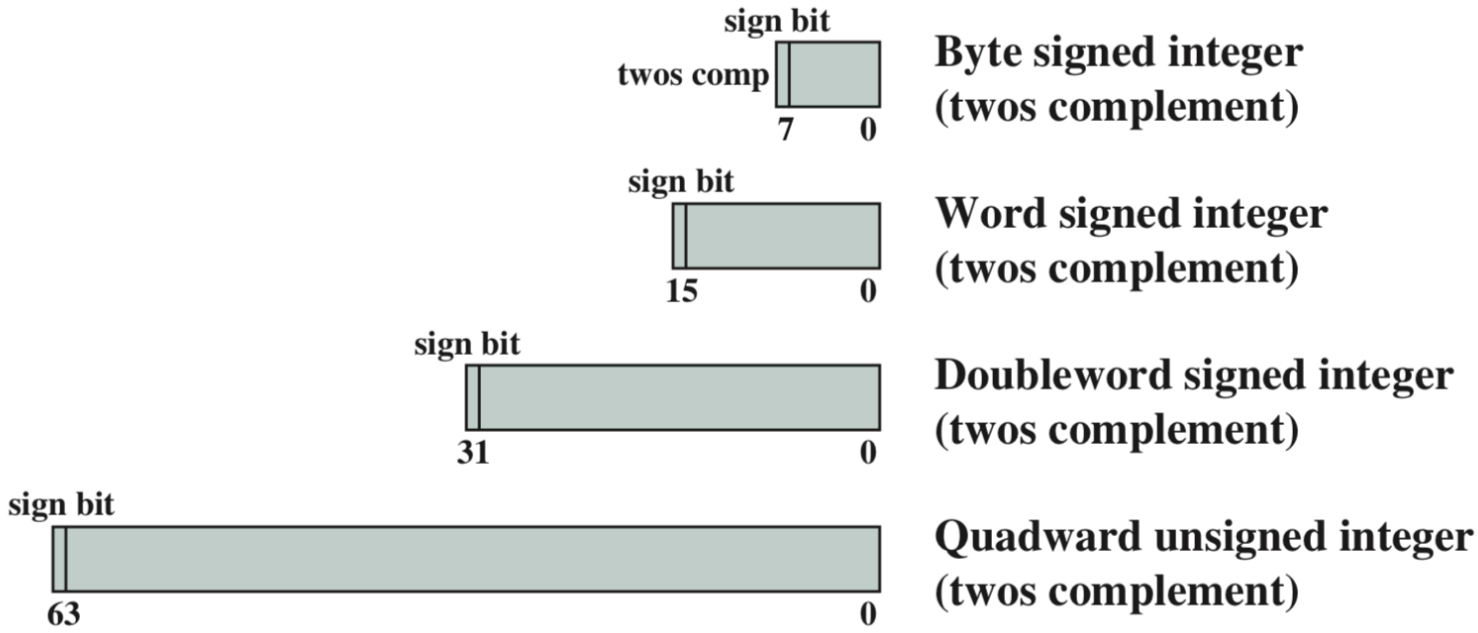
- The unsigned integers may be 16, 32, or 64 bits long.



Example 3b: x86 numerical data types.

(Signed integers)

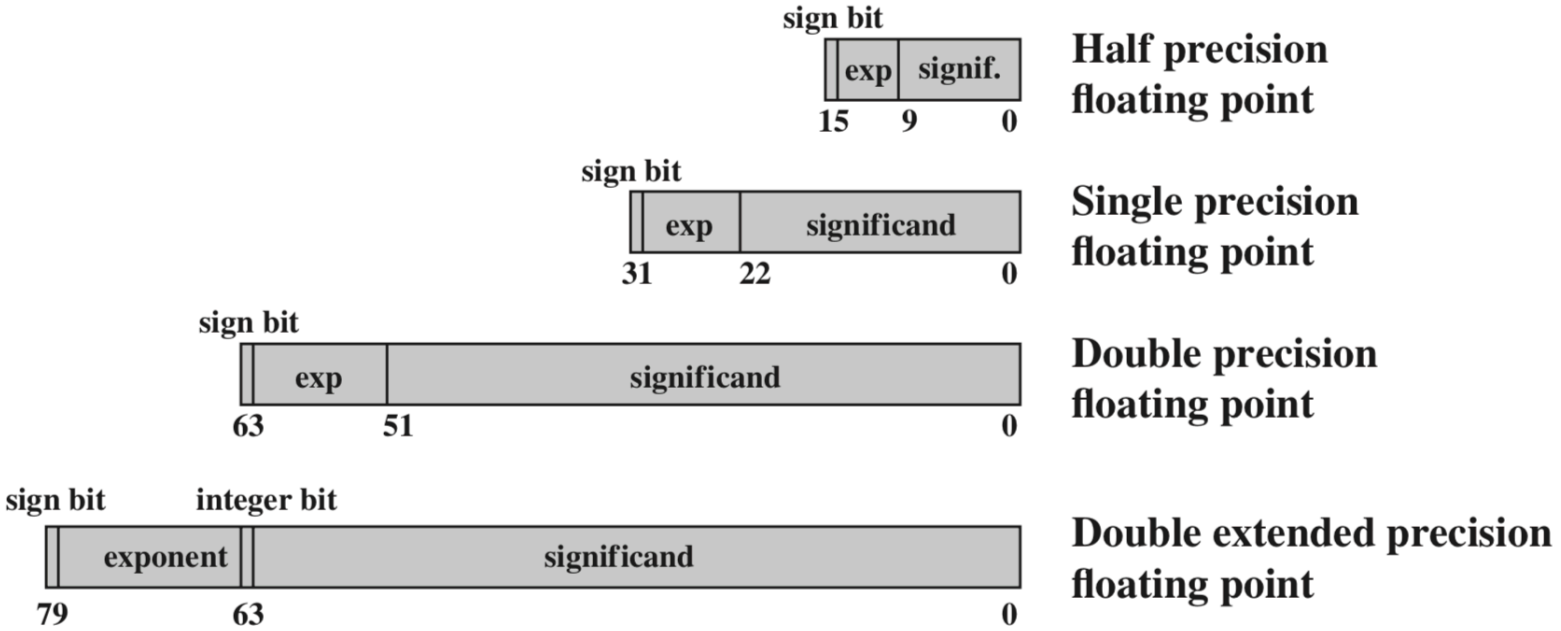
- The signed integers are in two's complement representation and may be 16, 32, or 64 bits long.



Example 3c: x86 numerical data types.

(Signed integers)

- The floating-point type actually refers to a set of types that are used by the floating-point unit and operated on by floating-point instructions → IEEE 754 standard.



4 Characters

- A common form of data is text or character strings.
- Today, the most commonly used character → *United States as the American Standard Code for Information Interchange* (ASCII).
- Another code used to encode characters is the *Extended Binary Coded Decimal Interchange Code* (EBCDIC); used on IBM mainframes.

4 Logical Data

- Normally, each word or other addressable unit (byte, halfword, and so on) is treated as a single unit of data → n-bit unit.
- When data are viewed this way, they are considered to be logical data.
- Two advantages to the bit-oriented view:



- ❑ Memory used efficiently to store an array of Boolean or binary data items for either values 1 (*true*) and 0 (*false*).
- ❑ Easy to manipulate the bits of a data item.

Module 4
Instruction Set Architecture (ISA)

4.1 Introduction

4.2 Machine Instruction Characteristics

4.3 Types of Operands

4.4 Addressing Modes

4.5 Instruction Formats

4.6 Summary

- Overview
- Immediate Addressing
- Direct Addressing
- Indirect Addressing
- Register Addressing
- Register Indirect Addressing
- Displacement Addressing

4 Overview

- Two issues arise in specifying the operands addresses and operations of instructions:

How the address of an operand is specified?

How the bits of an instruction are organized?

- A variety of addressing techniques has been employed to be able to reference a large range of locations in main memory or, for some systems, virtual memory.

Notations:

A = contents of an address field in instruction.

R = contents of an address field in instruction that refers to a register.

EA = actual (effective) address of the location containing the referenced operand.

(X) = contents of memory location X or register X.

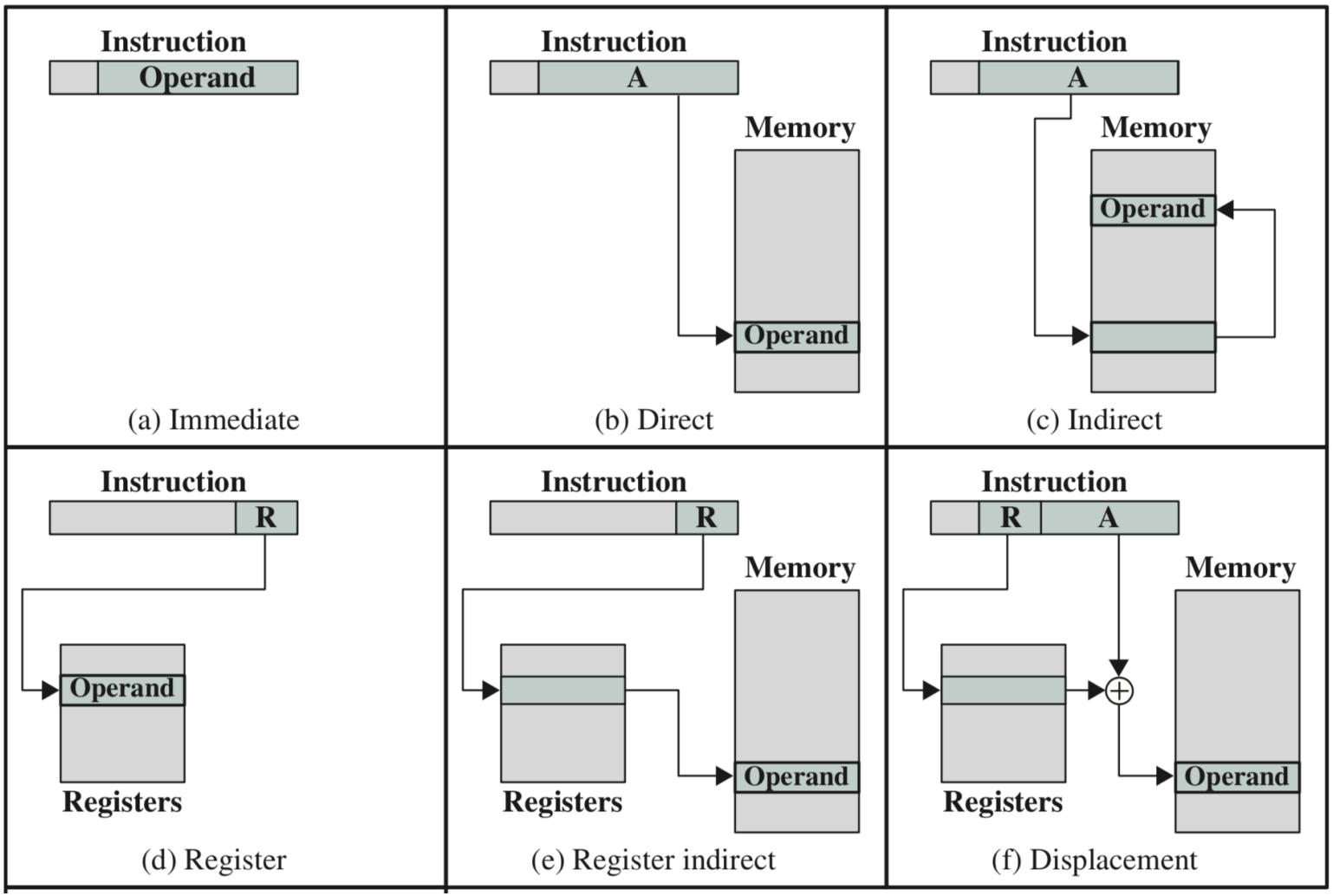
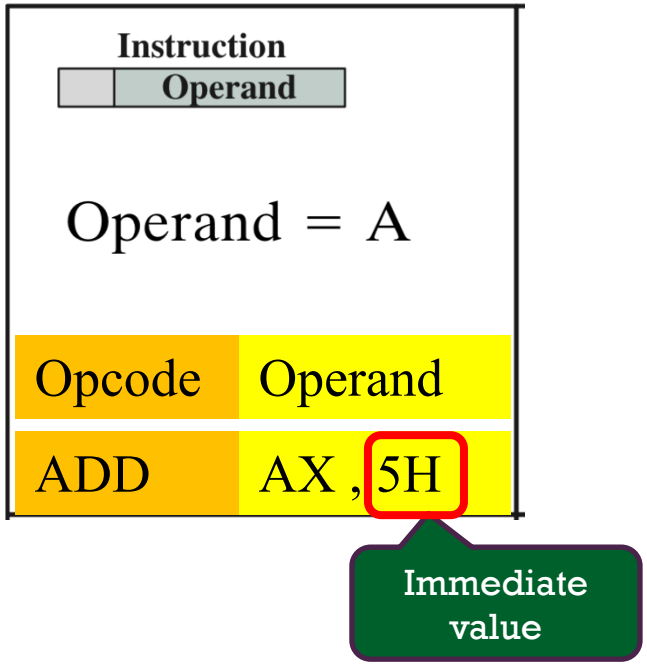


Figure: Addressing modes.



4

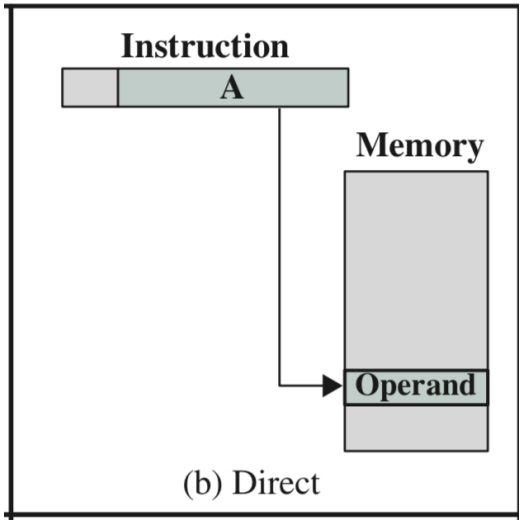
(a) Immediate Addressing

- The simplest form of addressing, in which the operand value is present in the instruction.
- Can defines and uses constant or set initial values of variables.

Saving one memory or cache → No memory reference other than the instruction fetch is required, (faster).

The size of the number is restricted to the size of the address field.

(b) Direct Addressing



$$EA = A$$

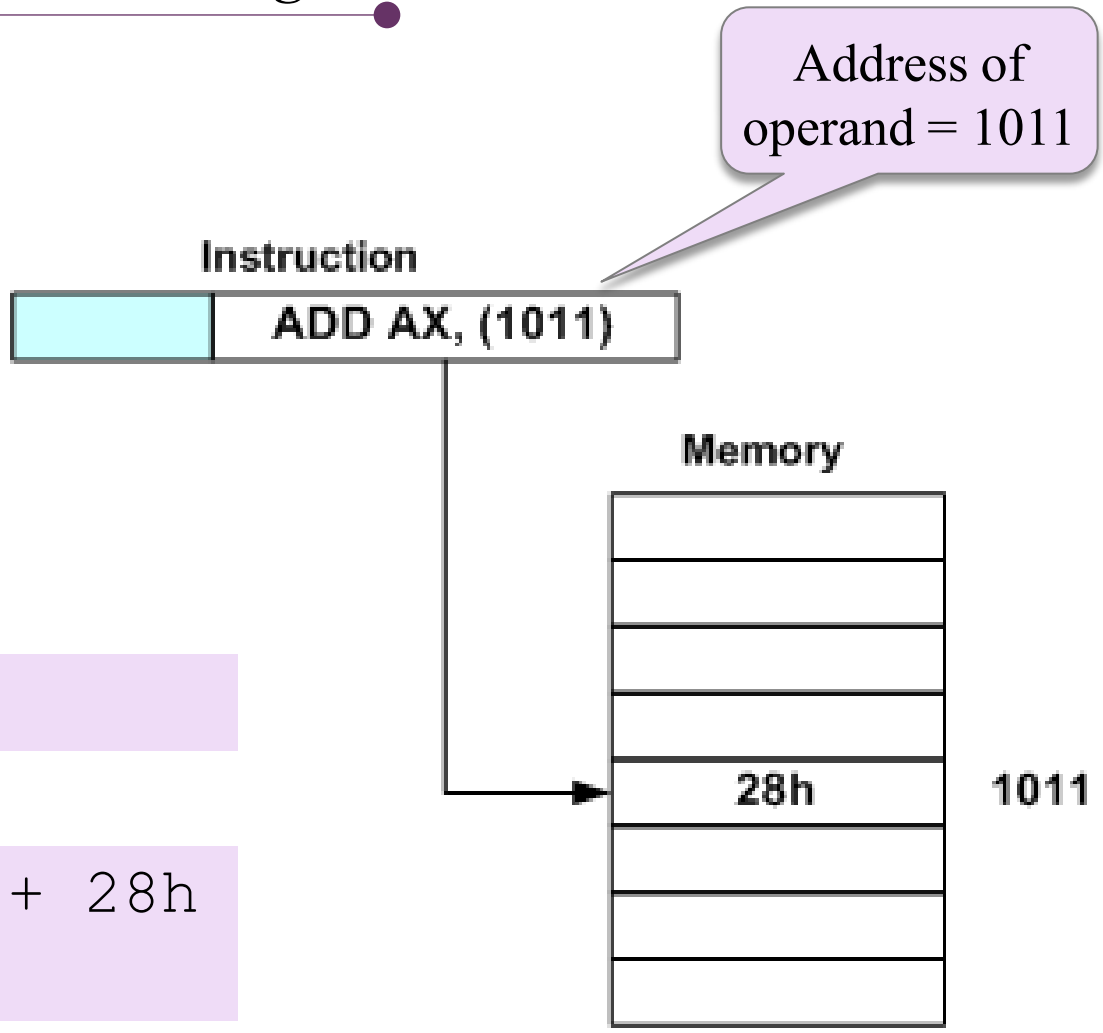
- The address field contains the *Effective Address* (EA) of the operand.

```
add     AX , count
add     AX , (1011)
```

Requires only one memory reference and no special calculation.

It provides only a limited address space.

Example 4: *Direct addressing.*




```
AX = 10h
```

```
AX = 10h + 28h
   = 38h
```

4

Activity: *Visual Studio*.



- Retype the next following codes in Example 5 – 12
- Then proceed with these 2 tasks:
  - (A) Execute ; [Start Without Debugging]
  - (B) Debug ; F10

Example 5: *Direct addressing.*

```
.data

val1   byte  10h
array1 word  2210h, 11h
array2 dword 123h, 234h

.code

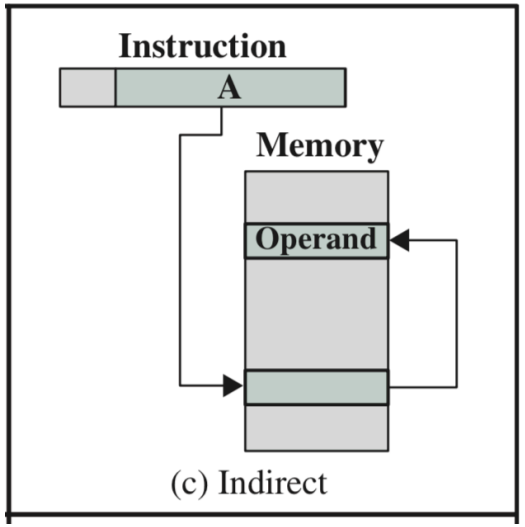
main PROC
      mov  al,val1
      mov  bx,array1
      mov  ecx,array2
      call DumpRegs
      exit
main ENDP
```

```
al  =         10h
bx  =       2210h
ecx = 00000123h
```

(*DumpRegs*)

```
EAX=7611ED10  EBX=7FFD2210  ECX=00000123  EDX=00401022
ESI=00000000  EDI=00000000  EBP=0012FF94  ESP=0012FF8C
EIP=0040102D  EFL=00000246  CF=0  SF=0  ZF=1  OF=0
```

(c) Indirect Addressing



$$EA = (A)$$

Look in A, find address (A), and look there for operand.

- Solution for the limitation of the address range in *direct addressing*
- Memory cell pointed to by address field (i.e., A) contains the address of the operand.

No particular advantage.

Three or more memory references could be required to fetch an operand; slower.

Example 6: *Indirect addressing.*

```
.data

array1 byte  10h
array2 word  123h,234h
array3 dword 123456h

.code

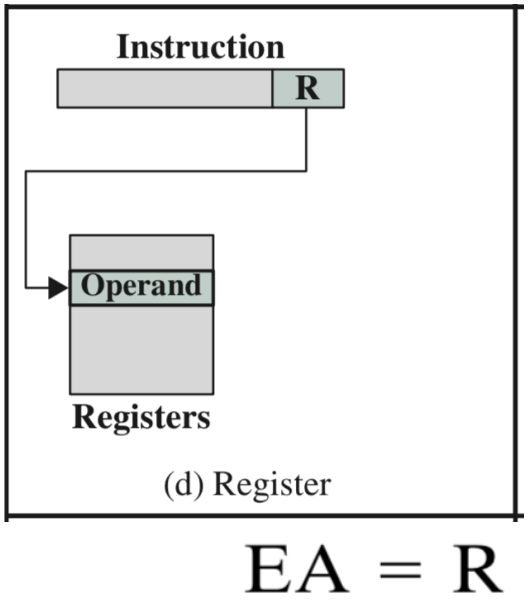
main PROC
      mov  bl,[array1]
      mov  cx,[array2]
      mov  edx,[array3]
      mov  ax,[array2+2]
      call dumpregs
      exit
main ENDP
```

```
bl  =       10h
cx  =     0123h
edx = 00123456h
ax  =     0234h
```

(*DumpRegs*)

```
EAX=00000234  EBX=00000010  ECX=00000123  EDX=00123456
ESI=00404004  EDI=00404008  EBP=0012FF94  ESP=0012FF8C
EIP=0040104C  EFL=00000246  CF=0  SF=0  ZF=1  OF=0
```

(d) Register Addressing



- Similar to *direct addressing*, except the address field refers to a register rather than a main memory address.
- The programmer need to decide which values should remain in registers and which should be stored in main memory.



No time consuming memory references needed → faster.



It has limited number of registers → provides a limited address space.

Example 7: *Register addressing.*

```
.data

.code

main PROC
        mov   eax,0
        mov   ebx,2000h
        mov   ecx,3000h

        mov   eax,ebx
        add   eax,ecx

        call dumpregs
        exit
main ENDP
```

```
eax = 00000000h
ebx = 00002000h
ecx = 00003000h

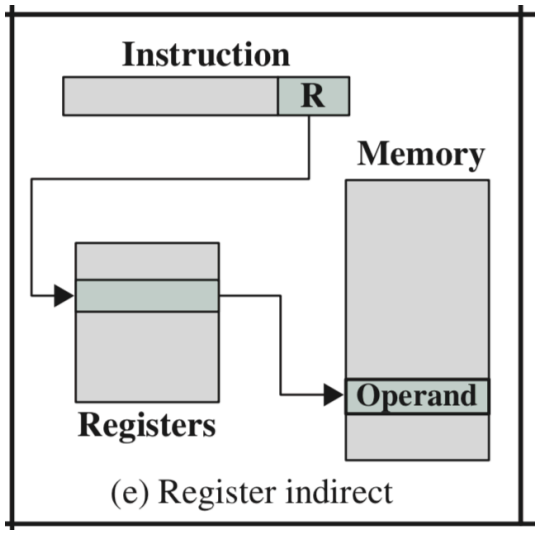
eax = 00002000h

eax = 00005000h
```

(*DumpRegs*)

```
EAX=00005000  EBX=00002000  ECX=00003000  EDX=00401005
ESI=00404004  EDI=00000000  EBP=0012FF94  ESP=0012FF8C
EIP=00401028  EFL=00000206  CF=0  SF=0  ZF=0  OF=0
```

(e) Register Indirect Addressing



$$EA = (R)$$

- Similar to *indirect addressing*.
- The advantages and limitations of *register indirect addressing* are basically the same as for *indirect addressing*.



Register indirect addressing uses one less memory reference than *indirect addressing*.

Example 8:

Register indirect addressing.

```
esi = 00404004h
edi = 00404008h
bl  =       10h
cx  =     0123h
edx = 00404004h
al  =       11h
```

```
.data
array1 byte  10h,11h,12h,13h
array2 word  123h,234h

.code

main PROC
      mov  esi,OFFSET array1
      mov  edi,OFFSET array2
      mov  bl,[esi]
      mov  cx,[edi]
      mov  edx,(esi)
      mov  al,[esi+1]
      call dumpregs
      exit
main ENDP
```

Address of the data that stored in register.

(*DumpRegs*)

```
EAX=00000011  EBX=00000010  ECX=00000123  EDX=00404004
ESI=00404004  EDI=00404008  EBP=0012FF94  ESP=0012FF8C
EIP=0040103D  EFL=00000246  CF=0  SF=0  ZF=1  OF=0
```

Example 8:

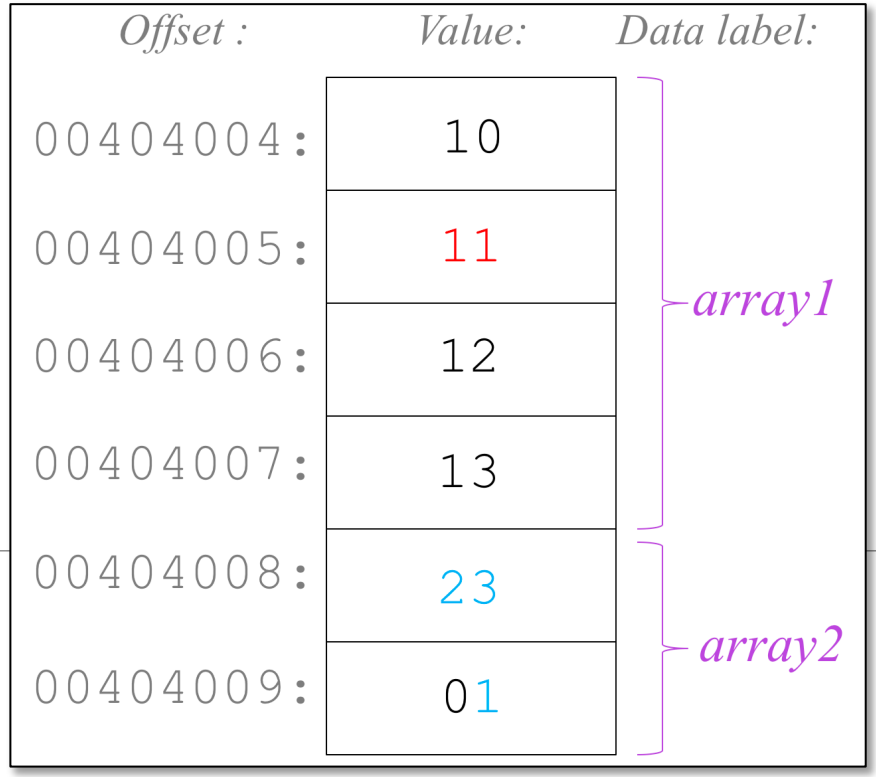
Register indirect addressing.

```
array1 byte  10h,11h,12h,13h
array2 word  123h,234h
```

```
esi = 00404004h
edi = 00404008h
```

```
mov  al,[esi+1]
```

```
al  =        11h
```



Example 9:

Register indirect addressing.

```
ebx = 00404000h
dl  =       24h
ebx = 00404001h
cl  =       55h
```

```
.data

.code

main PROC
        mov   ebx,404000h
        mov   dl,[ebx]
        inc   ebx
        mov   cl,[ebx]

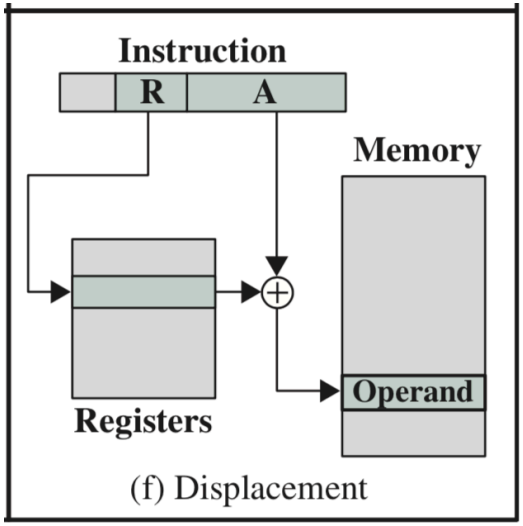
        call dumpregs
        exit
main ENDP
```

Offset :	*Value:*
00404000:	24
00404001:	55
00404002:	88

(*DumpRegs*)

```
EAX=00005000  EBX=00404001  ECX=00000055  EDX=00000024
ESI=00404000  EDI=00000000  EBP=0012FF94  ESP=0012FF8C
EIP=0040103D  EFL=00000202  CF=0  SF=0  ZF=0  OF=0
```

(f) Displacement Addressing



$$EA = A + (R)$$

3 common displacement addressing techniques:

- Relative addressing
- Base-Register addressing
- Indexed addressing

- A very powerful mode of addressing combines the capabilities of direct addressing and register indirect addressing.
- Address field hold two values:
  - A = base value
  - R = register that holds displacement

  (or vice versa)

(f.1) Relative Addressing

- Also called PC-relative addressing, the implicitly referenced register is the *Program Counter* (PC).

$$R = PC$$

$$EA = A + (PC)$$

- The next instruction address (shown in PC) is added to the address field to produce the EA.

EA = A + (PC)

- A = displacement ??
- EA = 00000028
- (PC) = 00000039

A = EA – (PC)

A = EA – (PC)
= 28 – 39
= FFFFFFEF (or -17 in base-10)
= **EF**

–(ve) because its jumping backwards

(1) Relative addressing

```
00000014   B9 00000000          mov ecx,0
00000019   BA 00000000          mov edx,0
0000001E   B8 00000000          mov eax,0
00000023   B9 00000004          mov ecx,4
00000028                   L1:
00000028   66| 8B 98            mov bx,array2[eax]
           00000008 R
0000002F   83 C0 02             add eax,2
00000032   E8 00000000 E        call dumpregs
00000037   E2 EF                LOOP L1

                                exit
00000039   6A 00          *     push   +000000000h
```

PC = 39

PC=39: *Need to go to L1 which is in* 00000028

(f.2) Base-Register Addressing

$$EA = A + R$$

- A holds displacement.
- R holds pointer to base address.
- R may be explicit or implicit.

Example 11:

Displacement Addressing.

(2) Base-register addressing

```
.data
array1 byte 10h,11h,12h,13h
array2 word 123h,234h,345h,456h
array3 dword 123456h, 23456789h


.code
main PROC
     mov esi,offset array1
     mov ebx,10h

     mov ax,word ptr[ebx+esi]

     add ebx,esi
     mov cx,word ptr[ebx]

     call dumpregs
     exit
main ENDP
```

esi =*base address*

ebx = *displacement*

Example 11:

Displacement Addressing.

Base-register addressing

```
esi = 00404004h
ebx = 00000010h
 ax =       6789h
ebx = 00404014h
 cx =       6789h
```

```
.data
array1 byte 10h,11h,12h,13h
array2 word 123h,234h,345h,456h
array3 dword 123456h, 23456789h

.code
main PROC
      mov esi,offset array1
      mov ebx,10h

      mov ax,word ptr[ebx+esi]

      add ebx,esi
      mov cx,word ptr[ebx]

      call dumpregs
      exit
main ENDP
```

Offset :	*Value:* array3
00404014:	89
00404015:	67
00404016:	45
00404017:	23

(*DumpRegs*)

```
EAX=753E6789  EBX=00404014  ECX=00006789  EDX=00401005
ESI=00404004  EDI=00000000  EBP=0012FF94  ESP=0012FF8C
EIP=00401028  EFL=00000206  CF=0  SF=0  ZF=0  OF=0
```

(f.3) Indexed Addressing

$$EA = A + R$$

- A = base
- R = displacement
- Good for accessing arrays

Example 12:

Displacement Addressing.

(3) Indexed addressing

```
.data
array1 byte 10h,11h,12h,13h
array2 word 123h,234h,345h,456h
array3 dword 123456h, 23456789h

.code
main PROC
    mov eax,0
    mov ecx,4

L1:
    mov bx, array2[eax]
    add eax,2

    call dumpregs
    LOOP L1
    exit
main ENDP
```

Base

Displacement

```
mov bx,[array2+eax]
add eax,2
```

Example 12:

Displacement Addressing.

(3) Indexed addressing

```
.data
array1 byte 10h,11h,12h,13h
array2 word 123h,234h,345h,456h
array3 dword 123456h, 23456789h

.code
main PROC
    mov eax,0
    mov ecx,4

L1:
    mov bx, array2[eax]
    add eax,2

    call dumpregs
    LOOP L1
    exit
main ENDP
```

```
eax = 00000000h
ecx = 00000004h

 bx =     0123h
eax = 00000002h

ecx = 00000003h
```

dec ecx

mov bx,[array2+eax]

[00404008+00000000]
→ array2[00404008]

Example 12:

Displacement Addressing.

(3) Indexed addressing

```
.data
array1 byte 10h,11h,12h,13h
array2 word 123h,234h,345h,456h
array3 dword 123456h, 23456789h

 code
main PROC
   mov eax,0
   mov ecx,4

L1:
   mov bx, array2[eax]
   add eax,2

   call dumpregs
   LOOP L1
   exit
main ENDP
```

2nd looping

bx = 0234h

eax = 00000004h

ecx = 00000002h

dec ecx

mov bx,[array2+eax]

[00404008+00000002]
→ array2[0040400A]

Example 12:

Displacement Addressing.

(3) Indexed addressing

```
.data
array1 byte 10h,11h,12h,13h
array2 word 123h,234h,345h,456h
array3 dword 123456h, 23456789h

 code
main PROC
   mov eax,0
   mov ecx,4

L1:
   mov bx, array2[eax]
   add eax,2

   call dumpregs
   LOOP L1
   exit
main ENDP
```

3rd looping

bx = 0345h

eax = 00000006h

ecx = 00000001h

dec ecx

```
mov bx,[array2+eax]
```

[00404008+00000004]
→ array2[0040400C]

Example 12:

Displacement Addressing.

(3) Indexed addressing

```
.data
array1 byte 10h,11h,12h,13h
array2 word 123h,234h,345h,456h
array3 dword 123456h, 23456789h

.code
main PROC
    mov eax,0
    mov ecx,4

L1:
    mov bx, array2[eax]
    add eax,2

    call dumpregs
    LOOP L1
    exit
main ENDP
```

4th looping

Exit looping

```
bx =      0456h
eax = 00000008h

ecx = 00000000h
```

dec ecx

```
mov bx,[array2+eax]
```

```
[00404008+00000006]
→ array2[0040400E]
```

The `DumpRegs` for the program:

```
EAX=00000002  EBX=00000123  ECX=00000004  EDX=00000000
ESI=00404004  EDI=00404008  EBP=0012FF94  ESP=0012FF8C
EIP=00401047  EFL=00000202  CF=0  SF=0  ZF=0  OF=0
```

```
EAX=00000004  EBX=00000234  ECX=00000003  EDX=00000000
ESI=00404004  EDI=00404008  EBP=0012FF94  ESP=0012FF8C
EIP=00401047  EFL=00000202  CF=0  SF=0  ZF=0  OF=0
```

```
EAX=00000006  EBX=00000345  ECX=00000002  EDX=00000000
ESI=00404004  EDI=00404008  EBP=0012FF94  ESP=0012FF8C
EIP=00401047  EFL=00000202  CF=0  SF=0  ZF=0  OF=0
```

```
EAX=00000008  EBX=00000456  ECX=00000001  EDX=00000000
ESI=00404004  EDI=00404008  EBP=0012FF94  ESP=0012FF8C
EIP=00401047  EFL=00000202  CF=0  SF=0  ZF=0  OF=0
```

Table: Basic addressing modes.





Mode	Algorithm	Principal Advantage	Principal Disadvantage
Immediate	Operand = A	No memory reference	Limited operand magnitude
Direct	EA = A	Simple	Limited address space
Indirect	EA = (A)	Large address space	Multiple memory references
Register	EA = R	No memory reference	Limited address space
Register indirect	EA = (R)	Large address space	Extra memory reference
Displacement	EA = A + (R)	Flexibility	Complexity
Stack	EA = top of stack	No memory reference	Limited applicability

Module 4
Instruction Set Architecture (ISA)

4.1 Introduction

4.2 Machine Instruction Characteristics

4.3 Types of Operands

4.4 Addressing Modes

4.5 Instruction Formats

4.6 Summary

- Overview
- Instruction Length
- Allocation Bits
- Variable-Length Instructions

4 Overview

- An instruction format defines the layout of the bits of an instruction, in terms of its constituent fields.
- An instruction format must include an opcode and, implicitly or explicitly, zero or more operands.
- The format must, implicitly or explicitly, indicate the addressing mode for each operand.
- For most instruction sets, more than one instruction format is used.

- Four common instruction formats:

(a) Zero-address instruction.



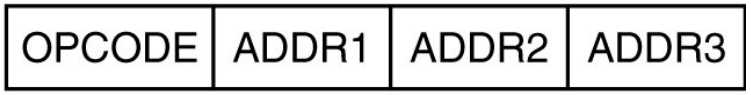
(b) One-address instruction



(c) Two-address instruction.



(d) Three-address instruction.



PDP-8 Instruction Format

Memory Reference Instructions

Opcode			D/I	Z/C	Displacement						
0		2	3	4	5						11

Input/Output Instructions

1	1	0	Device							Opcode	
0		2	3						8	9	11

Register Reference Instructions

Group 1 Microinstructions

1	1	1	0	CLA	CLL	CMA	CML	RAR	RAL	BSW	IAC
0	1	2	3	4	5	6	7	8	9	10	11

Group 2 Microinstructions

1	1	1	1	CLA	SMA	SZA	SNL	RSS	OSR	HLT	0
0	1	2	3	4	5	6	7	8	9	10	11

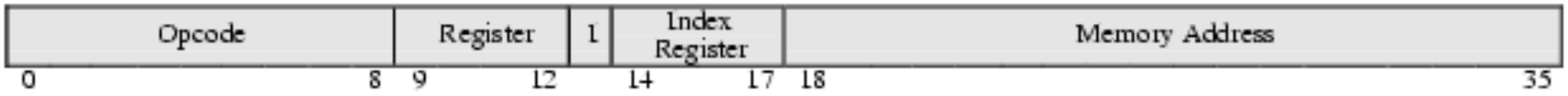
Group 3 Microinstructions

1	1	1	1	CLA	MI
0	1	2	3	4	

D/I = Direct/Indirect address
Z/C = Page 0 or Current page
CLA = Clear Accumulator
CLL = Clear Link
CMA = CoMplement Accumulator
CML = CoMplement Link
RAR = Rotate Accumultator Right
RAL = Rotate Accumulator Left
BSW = Byte SWap

- One of the simplest instruction designs for a general-purpose computer was for.
- Fixed length instruction format.
- The PDP-8 uses 12-bit instructions and operates on 12-bit words.
- There is a single general-purpose register, the accumulator.

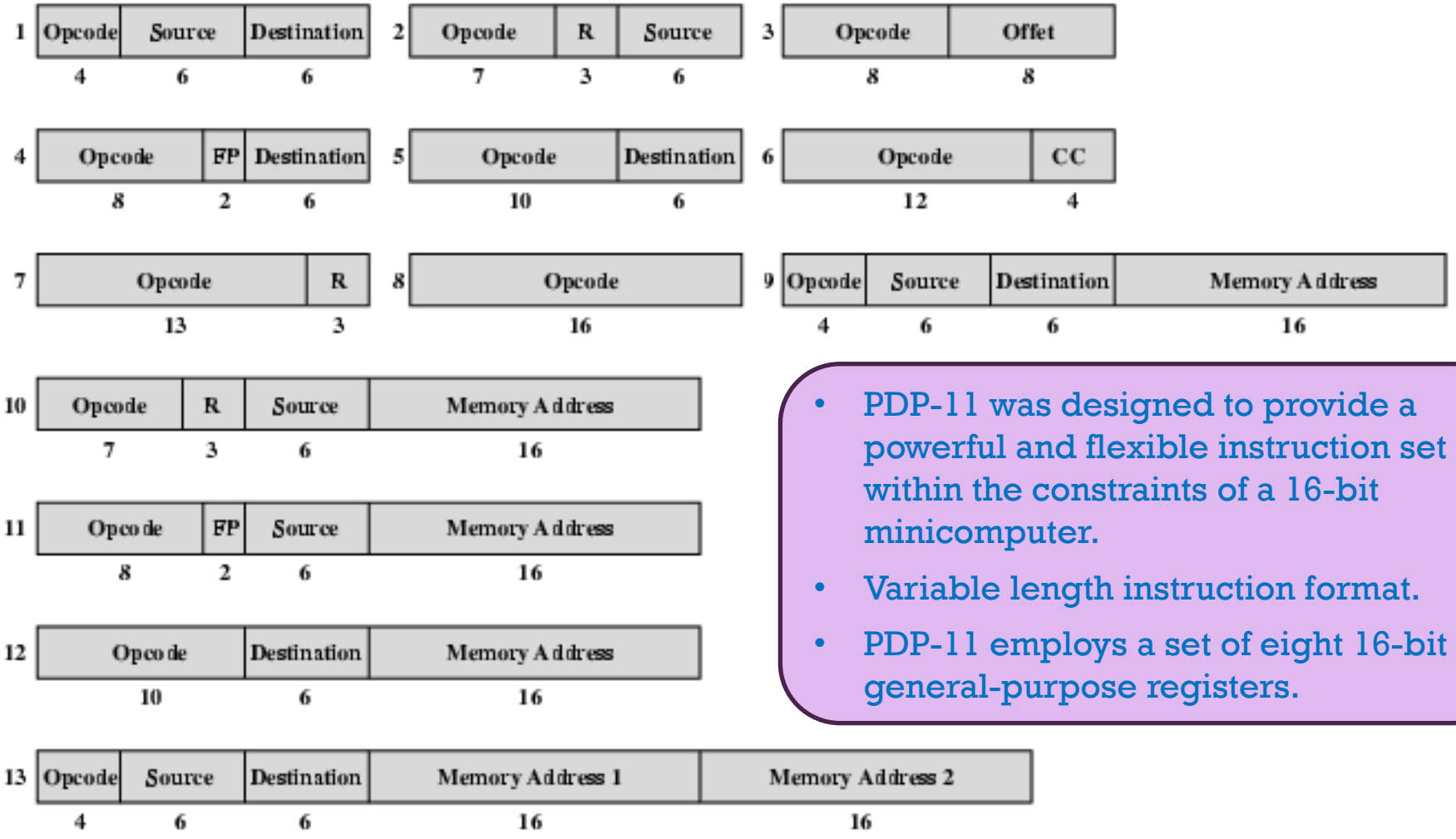
PDP-10 Instruction Format



I = indirect bit

- The PDP-10 has a 36-bit word length and a 36-bit instruction length.
- Fixed length instruction format.

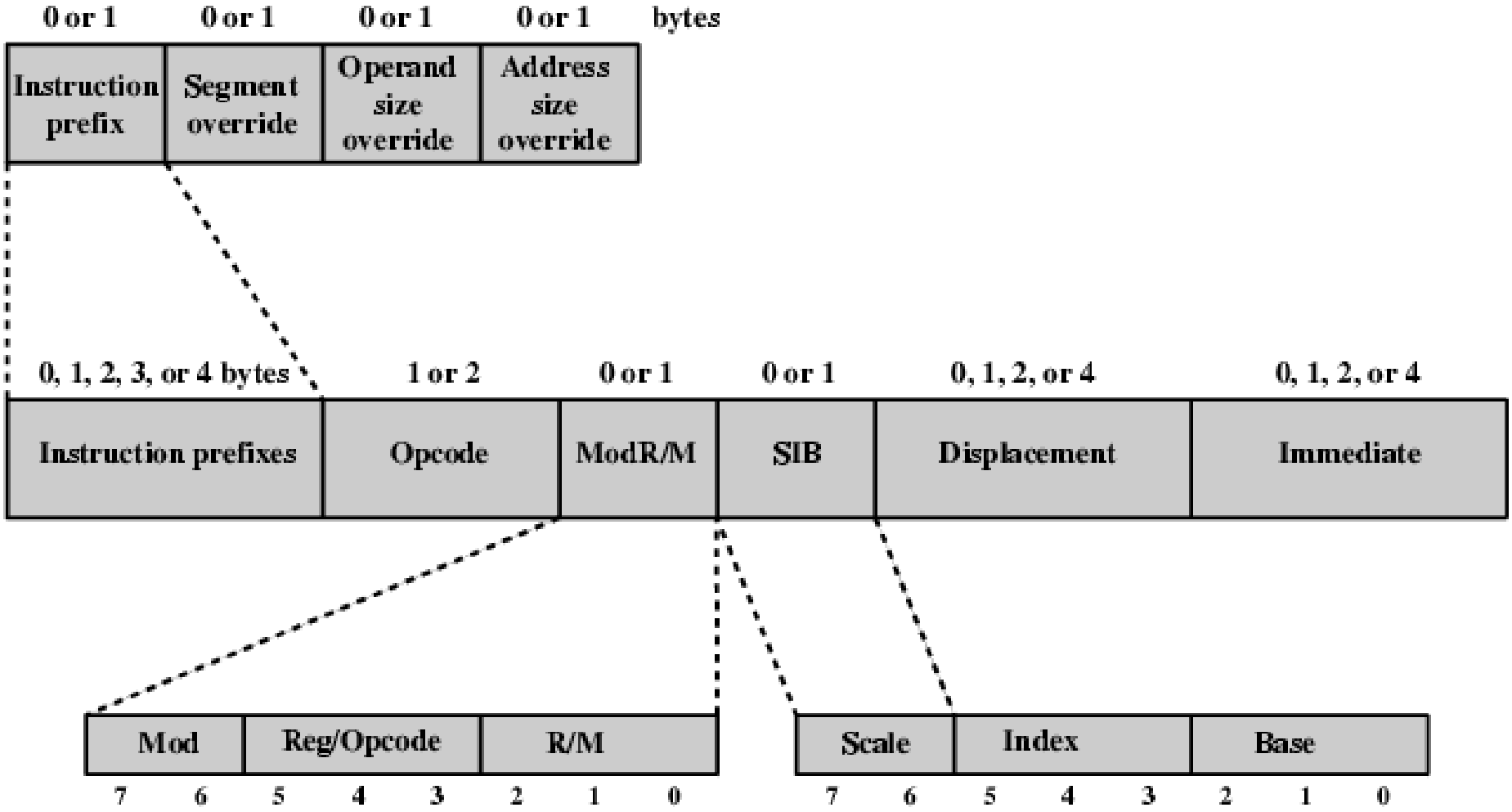
PDP-11 Instruction Format



- PDP-11 was designed to provide a powerful and flexible instruction set within the constraints of a 16-bit minicomputer.
- Variable length instruction format.
- PDP-11 employs a set of eight 16-bit general-purpose registers.

Numbers below fields indicate bit length
Source and Destination each contain a 3-bit addressing mode field and a 3-bit register number
FP indicates one of four floating-point registers
R indicates one of the general-purpose registers
CC is the condition code field

x86 Instruction Format



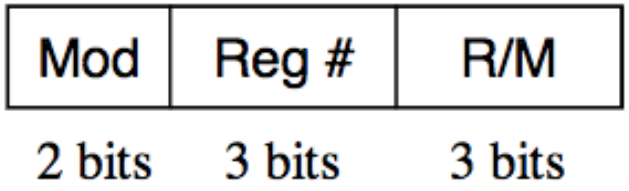
x86 Instruction Format

Opcode	ModR/M	SIB	displacement	Immediate
1 or 2 bytes	1 byte, If required	1 byte, If required	1,2 or 4 bytes If required	1,2 or 4 bytes If required

- Opcode: determine the action
- ModR/M: Addressing modes register/memory
- SIB: Scale-Index-Base
- Not all fields are present in all instr.
- If present, must be in the above order

x86 Instruction Format

ModR/M



- **Mod=00,**
  - First operand a register, specified by Reg #
  - Second operand in memory; address stored in a register numbered by R/M.
    - That is, Memory[Reg[R/M]]
  - Exceptions:
    - R/M=100 (SP): SIB needed
    - R/M=101 (BP): disp32 needed

x86 Instruction Format

Assembly Name	Reg #
EAX	000
EBX	001
ECX	010
EDX	011
ESP	100
EDP	101
ESI	110
EDI	111

x86 Instruction Format

- **Mod=01**, same as Mod 00 with 8-bit displacement.
  - Second operand: Memory[disp8+Reg[R/M].
  - Exception: SIB needed when R/M=100
- **Mod=10**, same as Mod 01 with 32-bit displacement
- **Mod=11**
  - Second operand is also a register, numbered by R/M.

x86 Instruction Format

Example: Add Instructions

- The first operand is the destination.
  - Can be register or memory
- The second operand is the source
  - Can be register or memory
- The two operands cannot be both memory.
- Action: dest += source

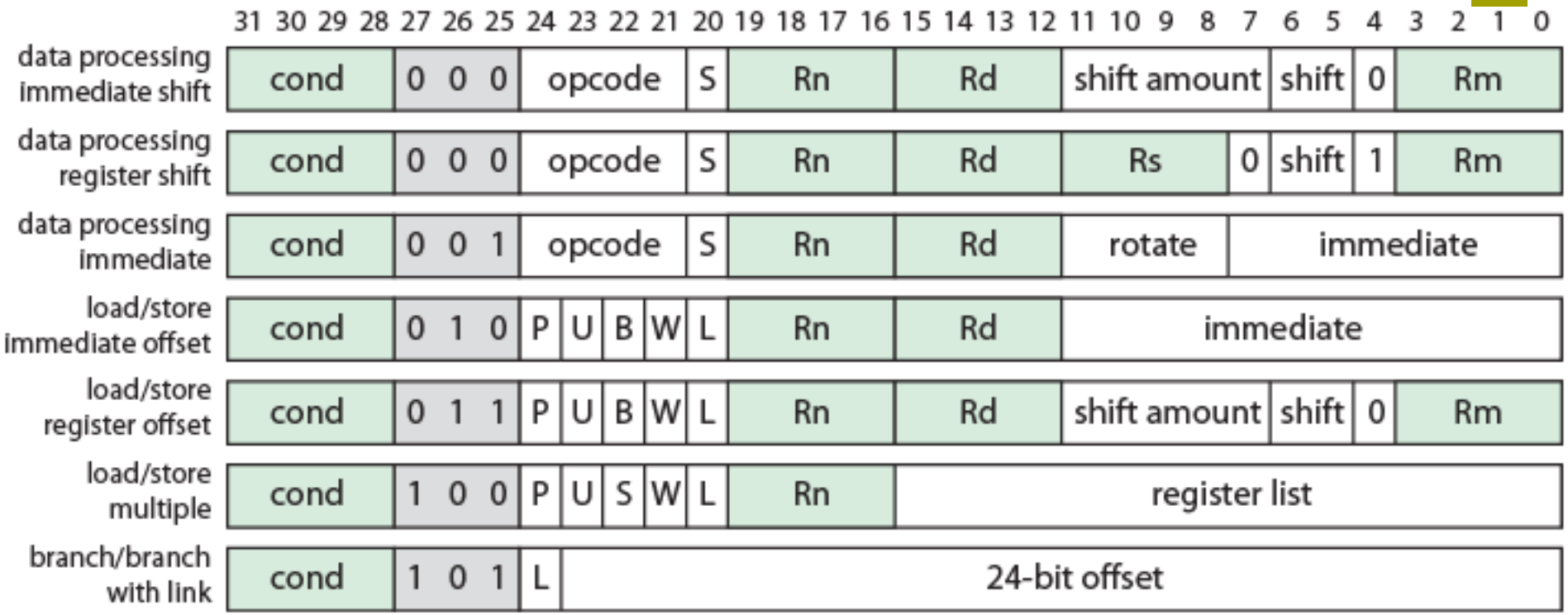
x86: add instruction

Opcode	Mnemonic	Operand(s)
00	add	mem8,reg8
01	add	mem16,reg16
		mem32,reg32
02	add	reg8,reg8
02	add	reg8,mem8
03	add	reg16,reg16
		reg32,reg32
03	add	reg16,mem16
		reg32,mem32

x86: add instruction

Opcode	Mnemonic	Operand(s)
04	add	AL,imm8
05	add	AX,imm16
		EAX,imm32
80	add	reg8,imm8
80	add	mem8,imm8

ARM Instruction Formats



- S = For data processing instructions, updates condition codes
- S = For load/store multiple instructions, execution restricted to supervisor mode
- P, U, W = distinguish between different types of addressing_mode
- B = Unsigned byte (B==1) or word (B==0) access
- L = For load/store instructions, Load (L==1) or Store (L==0)
- L = For branch instructions, is return address stored in link register

4 Instruction Length

- This decision affects, and is affected by:
  - memory size,
  - memory organization,
  - bus structure,
  - processor complexity, and
  - processor speed.
- Trade off between powerful instruction repertoire and saving space.

4 Allocation of Bits

- We've looked at some of the factors that go into deciding the length of the instruction format.

- An equally difficult issue is how to allocate the bits in that format, and the trade-offs here are complex.

- The following interrelated factors go into determining the use of the addressing bits:
  - Number of addressing modes – if indicated implicitly, certain opcodes might always call for indexing; if explicit – one or more mode bits will be needed.
  - Number of operands – typical instruction has 2 operands – uses mode indicator for operand addresses.
  - Register versus memory – single user register (accumulator), one operand address is implicit and consume no instruction bits; for multiple registers – a few bits are need to specify the registers.

4

- Number of register sets – have one set of general purpose registers with 32 or more registers in the set – for example sets of 8 registers only 3 bits are needed to identify the registers, opcode will implicitly determine which register set is being referenced.
- Address range – the range of addresses that can be referenced related to the number of bits.
- Address granularity – an address can reference a word or byte a the designer's choice – byte addressing is convenient for character manipulation.

4.6 Summary

4

- This chapter discussed on *what* an instruction set does by examining the types of operands and operations that may be specified by machine instructions.
- Described the various types of addressing modes common in instruction sets.
- Present an overview of essential characteristics of machine instructions.